

MarkSec: Capability-Aware Evaluation of Adversarial Attacks Against LLM Watermarks

Kairong Li Zhikun Zhang Xiao Ren Yunjun Gao*
 Zhejiang University, Hangzhou, China
 {kairong.li,zhikun.renx,gaoyj}@zju.edu.cn

Abstract

LLM watermarking has emerged as a promising solution for tracing the origin of LLM-generated content to help limit accidental or deliberate misuse. It faces various adversarial attacks: stealing, which recovers watermark-related information; scrubbing, which removes watermark signals from marked text; and spoofing, which forges text that a detector accepts as watermarked without actual watermark embedding. However, these attacks are often studied in isolation, leaving their connections unclear. Current evaluations also often lack shared detector calibration, metric definitions, and reporting semantics, making it difficult to compare the risks faced by different watermarks. In addition, attack success and text quality are measured separately, making it difficult to identify attacks that are both effective and quality-preserving.

To this end, we propose MarkSec, a general framework that unifies analyses of stealing, scrubbing, and spoofing. Building on this framework, we evaluate different attacks under a unified reporting protocol. We introduce a quality-constrained attack success metric to assess attack effectiveness and text quality jointly and support comparisons across watermark methods. Evaluations across representative watermark families, attack methods, LLMs, and datasets reveal three key findings. First, attacks that appear strongest by watermark removal alone can fall behind general rewriting under a criterion that jointly measures success and text quality. Second, general rewriting remains a strong baseline across watermark families, while its advantage over other scrubbing attacks varies by family. Third, in a case study of one watermark family, stealing-based scrubbers often underperform the best general-scrubbing baselines when text quality is required. These results show that apparent attack winners depend on text-quality constraints, attack generality, and capability assumptions.

Keywords

LLM Watermarks, Watermark Security, LLM Content Provenance

1 Introduction

Large language models (LLMs) are increasingly applied in software development [13, 26], education [33, 37], and online content production [20, 34]. As model outputs become more fluent and human-like, distinguishing LLM-generated text from human-written text becomes more difficult [1, 35], which creates provenance and misuse concerns. For instance, schools and universities have debated or restricted ChatGPT use because of concerns about plagiarism and academic dishonesty [7, 9]. AI-generated spam and low-quality synthetic content are increasingly polluting online platforms and search results [28]. These trends make tracing

LLM-generated content increasingly important in real-world LLM deployments [6]. *LLM watermarking* has therefore emerged as a promising solution because it embeds machine-verifiable signals during generation [16] and does not depend only on post-hoc detection, which is fragile under paraphrasing [17, 23].

LLM watermarking is useful only if it remains reliable and secure under adversarial manipulation. In practice, it faces three main threat types. *Stealing* attacks recover watermark-related information, such as key-dependent token preferences or detector scoring rules, and can use it to support stronger downstream attacks [14]. *Scrubbing* attacks remove or weaken watermark signals in marked text through paraphrasing, rewriting, or targeted editing [4, 31]. *Spoofing* attacks forge watermark-like outputs without actual watermark embedding, potentially creating false tracing signals [3].

Despite the broad attack surfaces, existing studies investigate these attacks mostly in isolation. Furthermore, these attacks are rarely evaluated with shared detector calibration, metric definitions, and reporting semantics. Prior studies use different models, watermark families, datasets, attack budgets, and reporting protocols, making it difficult to compare attack results across studies [19, 32]. In addition, the attack success rate and text quality are often measured separately, making it hard to identify attacks that are both effective and quality-preserving. As a result, it remains unclear how these attacks relate to one another, which attacks act as broad baselines versus narrow specialists, and which vulnerabilities matter most in practice.

Unified Framework. To address these issues, we study *stealing*, *scrubbing*, and *spoofing* as a connected attack space. Our key observation is that the three attack types are connected by how watermark-related information is obtained and reused. In particular, *stealing* can recover reusable artifacts, such as token preferences, detector statistics, or proxy distributions. The same artifacts can then support downstream *scrubbing*, which removes watermark evidence from existing text, or *spoofing*, which creates false attribution by generating watermark-like text. Motivated by this observation, we introduce a unified taxonomy that organizes these attacks by objective, capability, recovered artifact, and downstream use.

Experimental Evaluation. Building on this taxonomy, we evaluate adversarial attacks against LLM watermarks under a unified benchmark protocol. Our evaluation covers representative watermark families, attack methods, datasets, and LLMs. We report quality-constrained attack success, QSR, which jointly counts recorded removal and acceptance by a declared automatic quality gate. The main general-scrubbing benchmark focuses on three questions: how robust watermarks are overall, whether attack strength is broad or family-conditioned, and how rankings change

*Corresponding author.

under stricter Prometheus quality gates. Two further analyses examine Base-Instruct KGW differences across prompting protocols and the use of recovered watermark information for targeted scrubbing or spoofing. We also analyze quality-gate sensitivity across all six watermark families on common Llama/Qwen support.

Main Findings. Our evaluation shows that the reported attack ordering depends on the success definition and the automatic quality gate. On C4, SIRA leads raw removal, but LLMP leads quality-aware scrubbing; DIPPER remains the closest quality-aware alternative in several cells. Watermark removal alone can therefore overstate practical attack strength. Family-conditioned analysis further shows that LLMP is the strongest quality-aware general-scrubbing baseline on C4, with the narrowest margins under UW and KGW. LLMP achieves the highest mean QSR on C4 at all three Prometheus gates. The Instruct-Base QSR gap is smaller under wrapper-disabled prompting than under native prompting in the evaluated KGW settings. Among the stealing-based scrubbers, B4 leads QSR@3 in all four KGW settings; in spoofing, DE-MARK-Sp reaches 98–100% detector acceptance and 47–64% QSSR@3. The results call for a capability-aware reading: raw removal, quality-aware success, and recovered-artifact assumptions must be reported together. These findings show that apparent attack winners depend on text-quality constraints, watermark family, deployment format, and attacker capability assumptions.

MarkSec. We implement MarkSec as a modular and reusable tool for evaluating adversarial attacks against LLM watermarks. MarkSec provides common interfaces for watermark generation, attack execution, detector calibration, text-quality evaluation, and result reporting. It supports the joint study of *stealing*, *scrubbing*, and *spoofing* while keeping their capability assumptions explicit. The reporting pipeline groups general scrubbing, stealing-based scrubbing, and spoofing results by their access assumptions and downstream objectives.

2 Preliminaries

2.1 LLM Text Generation

Most modern text-generation LLMs are Transformer-based autoregressive models. Given an input prompt, the LLM first predicts a probability distribution over its vocabulary, which consists of all possible tokens. It then selects one token from this distribution and appends it to the current text. This process repeats until the model generates an end-of-sequence token or reaches a maximum length.

Formally, we view an LLM as a probabilistic generator parameterized by θ . Let $\mathcal{V}$ denote the token vocabulary, where $|\mathcal{V}|$ is the vocabulary size. Given a prompt sequence $\mathbf{x}^{(p)} = \{x_1, \dots, x_M\}$, the model generates a response sequence $\mathbf{x}^{(r)} = \{x_{M+1}, \dots, x_{M+T}\}$ token by token.

At each generation step t , the model takes the prompt and all previously generated tokens $\mathbf{x}_{<t} = \{x_1, \dots, x_{t-1}\}$ as input. It computes a logit vector $\mathbf{l}_t \in \mathbb{R}^{|\mathcal{V}|}$, where each entry gives an unnormalized score for one token in the vocabulary. The logits are then normalized by a softmax function to produce a probability distribution P_t over the vocabulary:

$$P_t(v | \mathbf{x}_{<t}) = \frac{\exp(\mathbf{l}_t[v])}{\sum_{v' \in \mathcal{V}} \exp(\mathbf{l}_t[v'])}, \quad v \in \mathcal{V}. \quad (1)$$

The next token x_t is selected from this distribution using a decoding strategy such as multinomial sampling, greedy decoding, or beam search.

2.2 LLM Watermarks

An LLM watermark is a machine-verifiable signal embedded into LLM-generated text for later provenance verification. A watermarked generation process should produce text that remains natural to human readers while carrying detectable statistical evidence. In general, LLM watermarks can be categorized by when and where the watermark signal is introduced. *Model-parameter watermarks* embed the signal into model-side parameters or learned watermark components, for example, by modifying parameters, fine-tuning behavior, or trigger-response patterns [21, 22]. *Inference-time watermarks* embed the signal during decoding by modifying logits, token probabilities, or the sampling rule without changing model parameters [6, 16]. *Post-hoc watermarks* introduce the signal after ordinary text generation, typically by editing, selecting, or filtering outputs according to a watermark rule [18].

In this paper, we focus on *inference-time* LLM watermarks. As described in Section 2.1, an LLM first produces token logits, then converts them into a probability distribution (*i.e.*, posterior), and finally samples the next token. Therefore, inference-time LLM watermarks can be further classified into three types according to where they intervene in this pipeline: logits-based, posterior-based, and sampling-based.

Logits-Based Watermarks. The general idea is to modify the model logits so that some key-dependent tokens become more likely to be sampled. At generation step t , a key-dependent rule assigns a bias to each candidate token, and the watermarked logits can be written as

$$\mathbf{l}_t^W[v] = \mathbf{l}_t[v] + b_{\xi,t}(v), \quad (2)$$

where $\mathbf{l}_t[v]$ is the original logit of token v , ξ is the watermark key, and $b_{\xi,t}(v)$ is a key-dependent bias. After softmax, tokens with larger positive bias receive higher sampling probability. The detector then tests whether the generated sequence contains more favored tokens than expected under unwatermarked generation.

KGW-style watermarks are representative examples of this class. They use the watermark key and previous context to partition candidate tokens into green and red sets, add a positive bias to green-token logits during decoding, and detect the watermark by testing whether the green-token count is statistically significant [16].

Posterior-Based Watermarks. Posterior-based watermarks operate on the normalized next-token distribution rather than on the pre-softmax logits. At generation step t , they apply a key-dependent transformation to the original distribution:

$$P_t^W = T_{\xi,t}(P_t), \quad (3)$$

where P_t is the original next-token distribution and $T_{\xi,t}$ is a key-dependent transformation. This transformation reallocates probability mass across tokens according to the watermark key while aiming to preserve the overall generation quality. The detector applies the same key-dependent rule to the observed sequence and tests whether the accumulated token statistics are more consistent with the watermarked distribution than with ordinary generation.

UW is a representative example of this class. It constructs a distribution-preserving transformation so that the marginal output distribution remains close to the original model distribution while still leaving a detectable keyed signal [10].

Sampling-Based Watermarks. These methods keep the probability distribution largely unchanged, but use a key-dependent sampling rule to choose the next token. Formally, a sampling-stage watermark can be written as

$$x_t = \text{Sample}_{\xi,t}(P_t), \quad (4)$$

where $\text{Sample}_{\xi,t}$ denotes a key-dependent sampling rule. The detector applies the same key-dependent rule to the generated sequence and accumulates a statistic that measures whether the observed tokens are more consistent with the watermarked sampling process than with ordinary sampling.

SynthID [6] is a representative example of this class. It uses tournament sampling during decoding. A key-dependent random function scores candidate tokens, and the sampler selects winners through pairwise token competitions, leaving a recoverable statistical signal in the generated sequence [6].

Watermark Detection. For all the above classes, watermark detection can be abstracted as a key-dependent hypothesis test. Given a text sequence $\mathbf{x}$ and a watermark key ξ , a detector computes a score $S_\xi(\mathbf{x})$ that measures how strongly the text matches the expected watermark signal. The detector then compares this score with a threshold τ :

$$D_\xi(\mathbf{x}) = \mathbf{1}[S_\xi(\mathbf{x}) \geq \tau]. \quad (5)$$

Different watermark methods instantiate S_ξ differently, such as a green-token count for logits-based schemes, a distributional consistency score for posterior-based schemes, or a key-dependent sampling statistic for sampling-based schemes. This unified detection view allows us to define attacks and evaluation metrics without depending on one specific watermark design.

3 Attacks Against LLM Watermarks

3.1 Threat Model

Attack Objectives. *Stealing* aims to recover reusable watermark-related information, such as token preferences, counting rules, green-list structure, or a surrogate distribution. *Scrubbing* aims to transform a watermarked text y_w into an attacked text y_a that is no longer detected as watermarked while preserving text quality. *Spoofing* aims to generate a forged text y_s that is accepted as watermarked without being produced by the genuine watermarking process.

Attacker Access. We assume the attacker does not know the model parameters θ or the secret watermark key ξ . The attacker may have black-box access to the watermarked generation API. Depending on the attack setting, the attacker may also receive the original prompt p , the generated watermarked text y_w , or victim-side token information such as probabilities, logits, or top- k scores. These access differences define the attack capabilities and determine which attacks are directly comparable in our evaluation. We do not assume access to hidden system prompts or proprietary serving templates. When prompt context is available, it refers to the user-facing original prompt.

3.2 Unified Framework

We organize attacks against LLM watermarks by their operational role. This avoids treating watermark attacks as a flat list of unrelated methods. Our key observation is that *stealing*, *scrubbing*, and *spoofing* are connected by how watermark-related information is obtained and used.

Stealing is an information-recovery objective. It recovers information that can later support stronger attacks. For example, a recovered green list, token-color map, or count table can be reused for targeted scrubbing or spoofing, while a distilled proxy distribution is used in this paper only for stealing-based scrubbing.

Scrubbing is a removal objective. It takes watermarked text as input and tries to erase the watermark signal while preserving text quality. We distinguish between direct scrubbing and stealing-based scrubbing. Direct scrubbing attacks rewrite, edit, or search over the given watermarked text without first recovering a reusable watermark-related artifact. Stealing-based scrubbing attacks first recover watermark-related information, such as token preferences or green-list structure, and then use this information to guide removal.

Spoofing is a forgery objective. It tries to create new text that is accepted as watermarked by the detector, even though the text was not produced by the genuine watermark process. Spoofing therefore attacks the attribution guarantee of the watermark rather than its removal robustness. In the methods we study, spoofing is usually downstream of stealing or inference because the attacker must recover enough watermark-related information to reproduce the watermark signal and trigger detection.

This relationship is illustrated in Figure 1. Direct scrubbing attacks form the main comparable removal setting because they operate on the same input type and do not require a prior recovery step. Stealing-based scrubbing and spoofing are analyzed separately because they rely on additional recovery steps or reusable artifacts. This separation keeps the main comparison focused while still capturing stronger attack paths in the full threat surface.

The taxonomy also defines the comparison boundary used in our experiments. Table 1 summarizes stealing-based methods by their stealing source, recovered artifact, and downstream use. Table 2 summarizes the direct scrubbing attacks used in the main comparison. This split keeps general scrubbing results comparable and still covers stronger attack paths that rely on additional capabilities.

Role Notation. Some attack papers support multiple downstream objectives. To avoid mixing different roles of the same method, we attach an objective suffix when a method appears in more than one part of the taxonomy. We use **St** for the stealing role, **Sc** for the scrubbing role, and **Sp** for the spoofing role. For example, **JSV-St** denotes the information-recovery stage of **JSV**, while **JSV-Sp** denotes its use for spoofing generation.

3.3 Stealing Attacks

Stealing attacks recover reusable watermark-related information before downstream use. The recovered artifact can be explicit, such as a green list or token-color map, or implicit, such as a context count table or proxy distribution. Its role is to expose information that later supports scrubbing or spoofing.

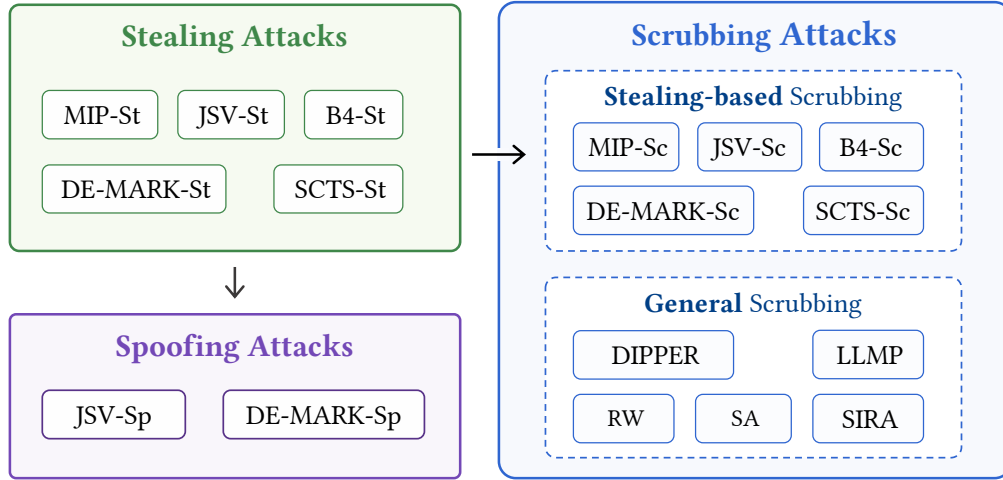


Figure 1: Attack taxonomy organized by adversarial objective: *Stealing*, *Scrubbing*, and *Spoofing*.

Table 1: Required capability of *stealing-based* attacks.

Method	Online	Prompt	Scores	Spoof
MIP [40]	✗	✗	✗	✗
SCTS [36]	✓	✓	✗	✗
DE-MARK [3]	✓	✓	✓	✓
JSV [14]	✗	✗	✗	✓
B4 [11]	✗	✓	✗	✗

SCTS-St [36]. Wu and Chandrasekaran propose SCTS for KGW-style watermarks, where token choices are biased by a hidden red-green partition. In its stealing role, SCTS estimates token-color information through black-box querying. The attacker queries the watermarked generator with adaptive query prompts and observes which candidate tokens are more likely to appear. These observations are used to infer a token-color map that approximates the hidden green and red sets. This recovered color map is the artifact later used by SCTS-Sc for targeted substitution.

JSV-St [14]. Jovanović et al. study whether hidden watermark behavior can be approximated from generated samples. In the stealing role, JSV builds context-dependent statistics from base and watermarked continuations. These statistics capture how the watermark shifts token choices under different contexts. The recovered count table is reusable because it summarizes watermark behavior beyond one attacked sample. This artifact can later guide JSV-Sc for removal or JSV-Sp for false attribution.

B4-St [11]. Huang et al. formulate B4 around a black-box view of the victim watermarked generator. Its stealing role is to learn a proxy distribution from victim-generated data. This proxy approximates how the watermarked system allocates probability mass over candidate continuations. The learned distribution becomes a reusable artifact for downstream rewriting. B4-Sc then uses this proxy to search for attacked text that reduces the watermark signal while preserving fidelity.

Table 2: Capability questions for *general scrubbing* attacks.

Method	Prompt	Scores	Locate	Local	Search
DIPPER [17]	✗	✗	✗	✗	✗
LLMP	✗	✗	✗	✗	✗
SIRA [4]	✗	✗	✓	✓	✗
RW [38]	✓	✗	✗	✓	✓
SA [2]	✓	✓	✓	✓	✗

MIP-St [40]. Zhang et al. formulate watermark stealing for green-list-style schemes as a mixed-integer optimization problem. The attacker uses the collected watermarked and reference text to infer which tokens are likely to belong to the hidden green list. The optimization stage produces a stolen green-list artifact. This artifact is reusable because it can guide future token replacement without repeating the full recovery process. MIP-Sc uses the recovered green list for targeted watermark removal.

DE-MARK-St [3]. Chen et al. design DE-MARK to infer watermark rules through token-level querying. In its stealing role, the attacker sends prompt-guided queries to the victim system and observes how candidate tokens behave under the hidden watermark. The query process estimates which tokens are favored by the watermark and may also recover parameter-level information. The resulting query evidence becomes a reusable guide for downstream attack.

3.4 Scrubbing Attacks

Scrubbing attacks form the main general-scrubbing comparison in MarkSec. These attacks take already watermarked text as input and try to remove the watermark signal while preserving text quality. They differ mainly in what information they use during rewriting, such as the text alone, the original prompt, proxy-model signals, or victim-side scores.

DIPPER [17]. As a generic paraphrasing baseline, DIPPER rewrites watermarked text into a semantically similar form. Its

intuition is that paraphrasing changes surface tokens and local token patterns, which can disrupt watermark signals embedded in the original wording. The attack does not use the watermark key, detector feedback, victim-side scores, or a recovered artifact. In MarkSec, DIPPER represents text-only direct scrubbing.

LLMP. Our LLMP baseline uses an LLM to rewrite the full watermarked continuation. It treats scrubbing as sequence-level rewriting: the model is prompted to preserve the meaning of y_w while producing a new expression. This attack can preserve high-level semantics because it rewrites the whole continuation at once. It does not use detector access, victim-side token scores, or a prior stealing stage. In MarkSec, LLMP represents general LLM-based direct scrubbing.

SIRA [4]. Cheng et al. propose SIRA to avoid rewriting the entire sequence. The attack uses self-information from a proxy model to locate spans that appear suspicious under the watermarked text distribution. It then rewrites only these high-suspicion spans. This targeted strategy aims to remove watermark evidence while changing fewer parts of the text. In MarkSec, SIRA represents proxy-guided direct scrubbing.

RW [38]. RW performs prompt-aware rewriting through a quality-guided search process. The attack generates candidate rewrites and keeps candidates that better balance watermark removal and text quality. The original prompt provides reference context during the search, helping the attack preserve the intended continuation. This makes RW different from one-shot paraphrasing baselines. In MarkSec, RW represents direct scrubbing with prompt-aware search and explicit quality control.

SA [2]. Chang et al. propose SA as a score-aware editing attack. The attack uses victim-side token information, such as probabilities, logits, or top- k scores, to identify tokens that contribute strongly to the watermark signal. It edits the text at the token level and searches for alternatives that preserve the local context. This gives SA finer control than full-sequence paraphrasing or span-level rewriting. In MarkSec, SA is a direct scrubber with stronger attack-time access because it uses victim-side token scores.

SCTS-Sc [36]. The scrubbing role of SCTS uses the token-color map recovered by SCTS-St. Given a watermarked continuation, the attack identifies tokens that are likely to be green under the hidden KGW-style partition. It then replaces these tokens with semantically similar alternatives that are less likely to be green. This directly targets the green-token count used by the detector. We classify SCTS-Sc as stealing-based scrubbing because its substitutions depend on the prior color-recovery stage.

JSV-Sc [14]. JSV-Sc uses the context-dependent statistics recovered by JSV-St to guide removal. The attack identifies token patterns that are likely to reflect watermarked generation under the recovered statistics. During rewriting or token selection, it avoids patterns that make the text look watermarked. This role uses stolen watermark behavior to reduce detector evidence in an existing continuation. We therefore treat JSV-Sc as stealing-based scrubbing.

B4-Sc [11]. B4-Sc uses the proxy distribution learned by B4-St to guide rewriting. The attack searches for continuations that reduce the watermark signal while staying close to the original text under the learned proxy. Its core trade-off is removal versus fidelity: aggressive changes may evade detection but damage text quality. The proxy distribution helps the attack navigate this trade-off in

a black-box setting. We classify B4-Sc as stealing-based scrubbing because its removal strategy depends on a reusable learned distribution.

MIP-Sc [40]. MIP-Sc uses the green list recovered by MIP-St. The attacker searches for edits that reduce the number of green-list tokens in the watermarked continuation. Because KGW-style detection depends on green-token overrepresentation, replacing green tokens with non-green alternatives can reduce the detection score. This scrubbing role is targeted and watermark-specific. We classify MIP-Sc as stealing-based scrubbing because it depends on the stolen green-list artifact.

DE-MARK-Sc [3]. DE-MARK-Sc uses query-derived watermark evidence to remove signals from an existing continuation. The attack identifies token choices likely favored by the hidden watermark rule. It then edits the text to avoid these choices while preserving meaning. This approach uses the recovered rule in the opposite direction of spoofing. We classify DE-MARK-Sc as stealing-based scrubbing because its removal strategy is guided by prior stealing.

Direct scrubbing attacks share a common goal: transforming watermarked text into attacked text without recovering a reusable watermark artifact. They differ in the side information and resources used for rewriting. All direct scrubbers in our study are detector-free; Table 2 asks whether an attack uses prompt context, victim-side scores, suspicious span/token localization, local edits, or iterative search.

3.5 Spoofing Attacks

Spoofing targets false attribution rather than watermark removal. A spoofing adversary generates text that is accepted by the victim detector as watermarked, even though the text was not produced by the actual watermark embedding process. In our taxonomy, spoofing is usually downstream of stealing because the attacker must reproduce enough of the watermark signal to trigger detection. We therefore analyze spoofing-related methods separately from direct scrubbing attacks.

JSV-Sp [14]. JSV-Sp uses context-dependent statistics recovered from JSV-St for detector-targeted generation. Instead of avoiding watermarked patterns, the attack biases generation toward them. The goal is to make unwatermarked text resemble outputs from the watermarked generator. This creates a false-attribution risk because the detector may accept forged text as authentic. We classify JSV-Sp as spoofing because it uses recovered watermark behavior to trigger detection.

DE-MARK-Sp [3]. DE-MARK-Sp uses the watermark rules recovered by DE-MARK-St during generation. The attacker favors tokens that the hidden watermark mechanism is expected to favor. This pushes the generated continuation toward the detector’s expected watermark pattern. Unlike DE-MARK-Sc, which avoids watermark-favored tokens, DE-MARK-Sp uses the same recovered information to imitate the watermark signal. We classify DE-MARK-Sp as spoofing because it forges text that can be falsely accepted as watermarked.

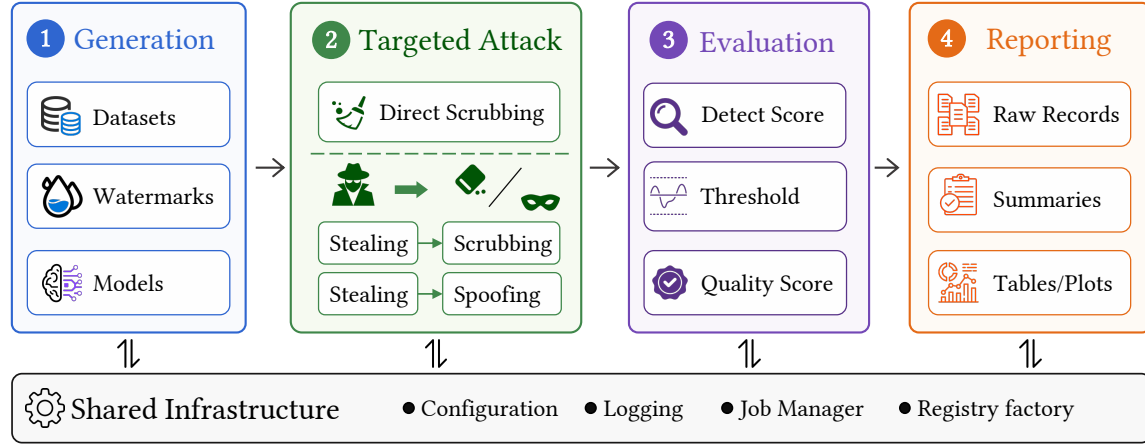


Figure 2: System overview of MarkSec.

4 MarkSec

In this section, we introduce MarkSec, a modular toolkit for evaluating adversarial attacks against LLM watermarks. MarkSec operationalizes the unified framework in Section 3 and turns it into an executable evaluation pipeline. MarkSec runs attack implementations and makes their results comparable under explicit capability assumptions. In particular, MarkSec jointly supports *stealing*, *scrubbing*, and *spoofing*, while keeping direct scrubbing attacks separate from stronger attacks that first recover reusable watermark-related artifacts. Figure 2 presents the system overview.

Existing studies on LLM watermark evaluation mainly focus on implementing watermark methods or reporting benchmark results [19, 24, 32]. Attack evaluation requires more than collecting attack scripts. Different watermark methods expose different generation interfaces, detector APIs, score directions, and threshold choices. Different attacks also require different inputs, access assumptions, recovered artifacts, and downstream goals. Without a shared toolkit, these differences can easily lead to results that are difficult to reproduce or compare across papers. MarkSec addresses this gap by providing a configuration-driven pipeline that loads common components, executes attacks under explicit capability assumptions, and reports attack success together with text quality.

Modules. MarkSec consists of four main modules: input and generation, attack orchestration, detection and metrics, analysis and reporting.

Input and Generation. This module loads prompt datasets, target LLMs, watermark methods, and generation configurations. It builds on existing watermark implementations and wraps them into a common generation pipeline. For each run, it records the prompt, the watermarked text, the watermark configuration, and the decoding metadata needed by later modules. This standardized input format decouples downstream attacks and metrics from watermark-specific implementation details.

Attack Orchestration. This module executes attacks according to their capability assumptions. Direct scrubbing attacks take watermarked text as input and produce attacked text under a shared removal protocol. Stealing-based methods first run a recovery stage,

such as querying, estimation, or distillation, and then pass the recovered artifact to downstream scrubbing or spoofing. The module records whether an attack uses the original prompt, victim-side scores, helper models, or reusable recovered artifacts. This makes the comparison boundary explicit at execution time rather than leaving it implicit in separate attack scripts.

Detection and Metrics. This module applies the detector associated with each watermark method to natural, watermarked, and attacked outputs. Because detectors may produce scores with different scales or directions, MarkSec normalizes score direction when needed and supports shared threshold calibration. The module computes detection metrics, attack success metrics, text-quality metrics, and quality-constrained attack success. This module lets MarkSec compare attack success together with text quality.

Analysis and Reporting. This module stores per-sample records, per-run summaries, metric details, and aggregate exports. The raw records preserve prompts, watermarked text, attacked text, detector outputs, quality metrics, and attack-specific metadata. The summary reports aggregate detection results, quality results, sample statistics, and the full run configuration. MarkSec also maintains a registry-backed reporting layer that supports large-scale tracking, analysis scripts, and paper-table exports. This reporting design keeps raw experimental artifacts available while allowing higher-level benchmark views to be regenerated.

Modular Design. MarkSec is organized around configuration-loaded wrappers for attacks, watermark generators and detectors, datasets, metrics, and experiment runners. New attacks are integrated through a shared attack interface and registered in an attack factory. This interface supports preparation, optional recovery or learning, editing, and saving or loading learned artifacts. Watermark generation and detection are loaded through wrapper interfaces, while datasets and metrics are managed through registries and metric managers. This design does not require every component to share the same internal implementation. Instead, it provides a common boundary where heterogeneous components can be loaded, executed, and reported through the same pipeline.

Supported Workflows. MarkSec supports three main workflows. First, it supports unified general-scrubbing evaluation. In

this workflow, comparable removal attacks are run under shared settings and summarized in a common report. Second, it supports capability-aware analysis of stronger attacks. In this workflow, attacks that first recover watermark-related information are evaluated together with their recovery assumptions, recovered artifacts, and downstream use. Third, it supports plug-and-play extension. Researchers can add new watermark methods, attack methods, models, datasets, or metrics while reusing the same generation, detection, and reporting pipeline.

Using MarkSec. Watermark designers can use MarkSec to assess whether a watermark is vulnerable to direct scrubbing or to stronger attacks that first recover watermark-related information. Attack researchers can use MarkSec to compare new attacks against existing baselines under a unified reporting protocol. Practitioners can use MarkSec to interpret watermark scrubbing success together with output utility, so that watermark removal is not mistaken for practical attack success when the resulting text is no longer useful. Because MarkSec separates direct scrubbing from stronger stealing- and spoofing-related methods, its reports make clear which results depend on additional attacker capabilities. This makes MarkSec both the implementation vehicle of our evaluation and a reusable toolkit for future studies of LLM watermark robustness.

5 Experimental Setup

5.1 Evaluation Settings

LLM Watermarks. We conduct experiments on six representative watermark schemes, including three logits-based schemes, KGW, Unigram, and SIR; two posterior-based schemes, UW and DIP; and one sampling-based scheme, SynthID. DIP is a watermark family and is unrelated to the DIPPER paraphraser attack. For implementation, we build on the MarkLLM repository [24] where available and adapt their input-output format to our protocol.

Evaluated SynthID Implementation. We evaluate MarkLLM’s SynthID-Text implementation in non-distortionary mode with the mean detector. The detector averages keyed g -values over channels and eligible completion n -grams, excluding repeated contexts and positions from EOS onward. Appendix F specifies its configuration and the cross-watermark sensitivity analysis.

Attacks. We include five general scrubbing attacks in the main experiments: DIPPER, LLMP, RW, SA, and SIRA. These attacks cover paraphrasing, LLM rewriting, quality-guided search, score-aware editing, and targeted proxy-guided rewriting. We further include five stealing-based methods in the stronger-capability analysis: MIP, SCTS, DE-MARK, JSV, and B4. Their objectives and capability assumptions are summarized in Section 3. For SCTS, we implement the attack in PyTorch [25]; for the remaining attacks, we adapt the authors’ official implementations to the benchmark settings. Detailed attack defaults are reported in Appendix D.

Target LLMs. The main benchmark evaluates three widely used open-source LLM families: Llama 3.1 8B Instruct [8], Qwen2.5 7B Instruct [29], and Mistral 7B Instruct v0.3 [12]. These targets cover different tokenizer and model families while keeping the matrix computationally tractable. All local models are loaded from fixed

checkpoints to avoid inconsistencies caused by remote model updates. For the instruction-tuning sensitivity study, we additionally compare Qwen2.5-7B Base and Instruct checkpoints under the same named KGW settings.

Prompt Datasets. We conduct experiments on three prompt datasets: C4 [30], Dolly-15K [5], and MMW BookReport, the book-report prompt set from Mark My Words (MMW) [27]. We select 100 prompts per dataset. To better match instruction-tuned models, we prepend an instruction-style continuation prompt to each C4 sample.

Sample Universe. The main matrix uses 26,096 recorded outputs from 261 runs conducted in April 2026. Each attack block is computed on its own completed-output population; execution completion includes empty or otherwise invalid outputs. Generated source texts and detector thresholds can differ across attack runs, so cross-method comparisons aggregate configuration-level rates. Within a run, paired detection refers to the same sample before and after attack. The records include 35 samples with insufficient context for SIR detection and two empty SA outputs with unavailable after-detection. Structural exclusions are marked “N/A”; undefined measured rates are marked “–”.

5.2 Evaluation Metrics

Robustness Metrics. True positive rate (TPR) is the fraction of watermarked texts detected as watermarked. False positive rate (FPR) is the fraction of unwatermarked texts incorrectly detected as watermarked. Detector calibration targets an FPR of 1%; results use the decision and threshold recorded for each run. This is a calibration target, not a separately measured test-set FPR. When a detector’s raw score uses the opposite direction, we invert the decision direction so that all reported results share the same interpretation. Let C contain the completed-output sample IDs of a run. Define $S_i = 1$ only when both detector states are present and sample i moves from detected to undetected; otherwise it earns no removal credit. Let $C_{+,pair}$ contain samples with both states observed and a positive before-state, and let P_i be the recorded Prometheus score. We compute

$$ASR = \frac{\sum_{i \in C} S_i}{|C_{+,pair}|}, \quad (6)$$

$$QSR@u = \frac{\sum_{i \in C} \mathbf{1}[S_i = 1, P_i \geq u]}{|C|}. \quad (7)$$

Unavailable quality or detector evidence cannot pass the joint-success gate. ASR uses only before-positive samples with both detector states observed; it is undefined when this set is empty. TPR_b is the positive fraction among all available before-detection results, regardless of after-detection availability. ASR macro-averages and ranks use only cells with defined ASR; QSR macro-averages use all available cells, including those with zero detected inputs. Each included cell receives equal weight, and figure data report the contributing cell counts. The ASR–QSR difference therefore combines eligibility, measurement coverage, and the quality gate.

Text-Quality Metrics. We use Prometheus (Prom) [15] to assess output quality at thresholds 2, 3, and 4. BERTScore with roberta-large and PSP provide complementary similarity measures. QSR denotes removal jointly satisfying the specified automatic quality

gate; QSR@3 is the default. BERTScore and PSP compare full-text fields that include a shared prompt when present (Appendix E.2). Quality evaluation uses automatic scores; supplementary content screening uses deterministic rules and Codex-assisted source-output inspection, with no independent human annotation. Source adequacy and preservation are assessed separately: a faithful edit can retain an inadequate source.

5.3 Evaluation Matrix And Reporting Scope

Each benchmark run is defined by a tuple (D, M, W, A) , where D represents a prompt dataset, M a target LLM, W a watermark scheme, and A an attack method. To ensure readability and protocol control, the main text focuses on the canonical C4 slice. Additional Dolly-15K and MMW BookReport slices are included in the appendix to examine dataset dependence.

We report general scrubbing attacks as the main comparison surface because they share the same downstream objective: given watermarked text, produce an attacked text with weakened watermark evidence while preserving utility. Stealing- and spoofing-related methods are reported separately because they require additional capabilities, such as query access, query budgets, prebuilt corpora, recovered artifacts, or attack-side generation control. This shared-protocol comparison answers how robust watermark families are under general scrubbing. The stronger-capability analysis asks when recovered watermark-related information changes the risk picture through targeted scrubbing or spoofing.

6 Experimental Results

In this section, we answer five empirical questions.

- RQ1.** How robust are LLM watermarks under the shared general-scrubbing protocol?
- RQ2.** Are general scrubbing attacks broad quality-aware baselines or watermark-family specialists?
- RQ3.** How does Prometheus-based utility filtering change attack rankings?
- RQ4.** How do the Base-Instruct observations vary with prompting protocol?
- RQ5.** How does recovered watermark-related information change scrubbing and spoofing risk?

The first three questions use the shared general-scrubbing benchmark, where each attack receives watermarked text and outputs attacked text under the same reporting protocol. RQ1 measures robustness under this shared protocol. RQ2 separates broad quality-aware baselines from family-aligned specialists. RQ3 asks whether rankings survive Prometheus-gated utility filtering. RQ4 compares Base-Instruct outcomes under two prompting protocols. RQ5 studies whether recovered watermark-related information changes downstream scrubbing and spoofing risk. We report RQ5 separately from the general-scrubbing leaderboard because its access assumptions are different.

6.1 RQ1: Overall General-Scrubbing Robustness

We first evaluate general scrubbing on the canonical C4 slice. Table 3 reports pre-attack detectability, raw attack success, and

quality-constrained success for each watermark-victim-attack combination. We abbreviate pre-attack TPR as TPR_b . Higher ASR and QSR mean stronger attacks. Bold marks the row-wise best QSR. We use “N/A” for combinations excluded by the benchmark protocol. In this table, “N/A” appears for SA under selected Mistral settings because SA requires a tokenizer-aligned helper model, and our Mistral setup does not include a matching small helper model. The relevant distinctions are baseline detectability, paired detector evasion, and automatic-quality-gated success.

Two patterns stand out. First, attack success has to be interpreted relative to the pre-attack detector strength. Some watermark families start from much stronger detectability than others, so a high ASR on a weakly detectable setting is not equivalent to breaking a strongly embedded watermark. Second, no general scrubbing attack dominates all axes. SIRA leads the recorded C4 removal rate, while LLMP leads joint success under the automatic Prometheus gate. DIPPER remains the closest quality-aware alternative on C4.

Figure 3 visualizes the difference between the two reported metrics. On C4, SA illustrates the danger of using ASR alone. It often drives detector scores down, but fewer records receive joint credit under the automatic gate and joint-rate denominator. LLMP and DIPPER lose less of their apparent success when QSR is applied. Across dataset slices, attack rankings differ between ASR and QSR@3.

Finding 1. On C4, SIRA achieves the highest mean ASR, whereas LLMP achieves the highest mean QSR@3.

6.2 RQ2: Family-Specific Robustness Patterns

RQ2 asks whether general scrubbing attacks are broadly effective across watermark families or mainly strong when their mechanism aligns with a specific watermark. This question matters because overall averages can hide two different behaviors: a stable broad baseline and a narrow family specialist.

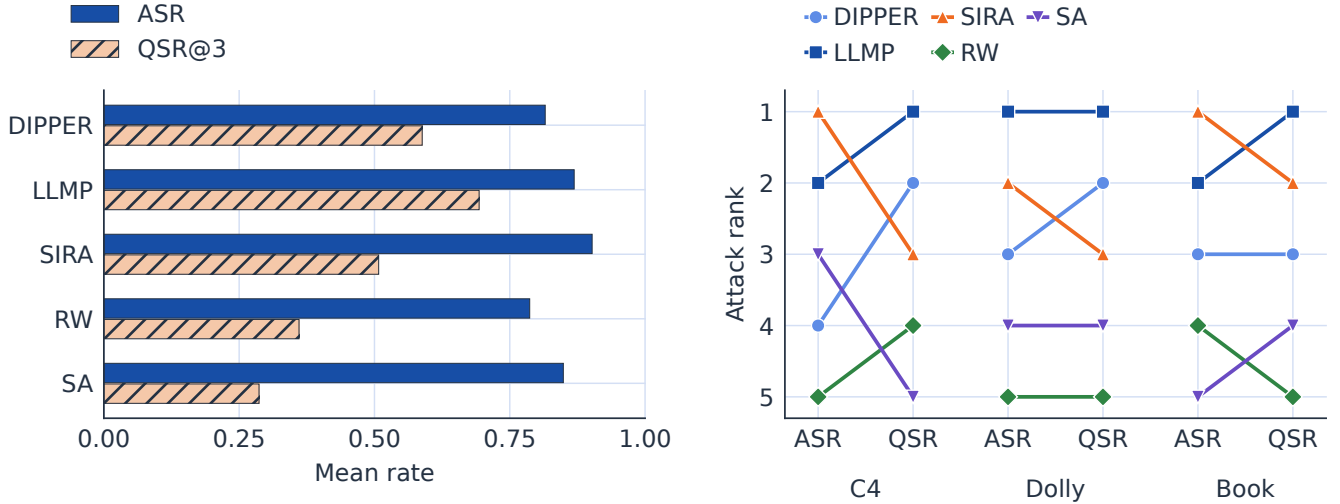
To answer RQ2, we read the C4 matrix by watermark family and summarize the interaction in Figure 4. The main result is that quality-aware attack rankings are less fragmented than raw removal suggests. LLMP has the highest mean QSR@3 for every C4 watermark family. The family-conditioned view still matters because the margin changes by family. UW and KGW leave DIPPER close to LLMP, while SynthID shows the widest LLMP lead. Unigram, SIR, and DIP fall between these two cases.

Figure 5 adds a supporting view over dataset-watermark cells. Unlike Figure 4, which summarizes family-level averages, this figure shows how stable each attack is across individual (D, W) conditions. Each point is one cell, and each box summarizes the QSR@3 distribution for one attack. LLMP has the highest quality-aware success distribution across these cells. Other attacks either sit lower overall or show wider variation.

This interaction view changes how we should interpret attack rankings. Overall averages identify LLMP as the strongest quality-aware attack on C4. Family-conditioned analysis explains where that lead is narrow and where it is robust. SIRA leads mean ASR on C4, while LLMP leads mean QSR@3 in every C4 watermark

Table 3: C4 general-scrubbing results across watermark families, victim models, and attacks.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		TPR _b ↑	ASR ↑	QSR ↑	TPR _b ↑	ASR ↑	QSR ↑	TPR _b ↑	ASR ↑	QSR ↑	TPR _b ↑	ASR ↑	QSR ↑	TPR _b ↑	ASR ↑	QSR ↑
KGW	Llama	0.93	0.78	0.63	0.92	0.67	0.57	0.93	0.82	0.42	0.96	0.61	0.35	1.00	0.46	0.26
	Qwen	0.96	0.75	0.61	1.00	0.88	0.72	0.96	0.91	0.74	0.99	0.62	0.26	0.95	0.68	0.33
	Mistral	0.94	0.88	0.71	0.97	0.82	0.78	0.94	0.89	0.44	0.95	0.81	0.39	N/A	N/A	N/A
Unigram	Llama	0.72	0.75	0.46	0.77	0.75	0.53	0.72	0.96	0.43	0.73	0.88	0.30	0.73	0.49	0.19
	Qwen	0.81	0.85	0.57	0.86	0.98	0.78	0.81	0.95	0.65	0.80	0.64	0.24	0.75	0.89	0.22
	Mistral	0.90	0.60	0.50	0.81	0.78	0.58	0.90	0.88	0.47	0.91	0.59	0.31	N/A	N/A	N/A
SIR	Llama	0.62	0.62	0.33	0.55	0.78	0.40	0.51	0.84	0.27	0.49	0.71	0.23	0.59	0.76	0.26
	Qwen	0.69	0.79	0.41	0.70	0.96	0.63	0.77	0.95	0.62	0.71	0.76	0.32	0.70	0.91	0.24
	Mistral	0.91	0.55	0.43	0.91	0.69	0.61	0.91	0.73	0.38	0.93	0.80	0.32	N/A	N/A	N/A
UW	Llama	0.90	0.93	0.71	0.85	0.92	0.75	0.90	0.96	0.47	0.87	0.78	0.48	0.90	0.73	0.24
	Qwen	0.84	0.99	0.68	0.83	1.00	0.76	0.84	1.00	0.71	0.75	0.93	0.37	0.82	1.00	0.26
	Mistral	0.95	0.96	0.84	0.94	0.93	0.82	0.95	0.96	0.43	0.98	0.84	0.50	0.95	0.99	0.46
DIP	Llama	0.57	1.00	0.49	0.72	0.90	0.61	0.57	0.93	0.37	0.70	0.89	0.40	0.75	0.97	0.19
	Qwen	0.82	0.94	0.59	0.81	0.96	0.72	0.82	0.96	0.69	0.78	0.88	0.34	0.86	0.99	0.32
	Mistral	0.85	0.94	0.68	0.84	0.95	0.76	0.85	0.92	0.35	0.85	0.85	0.37	0.78	0.99	0.46
SynthID	Llama	0.95	0.80	0.67	0.99	0.89	0.85	0.95	0.87	0.43	0.96	0.86	0.47	0.96	0.95	0.22
	Qwen	0.98	0.85	0.70	0.93	0.97	0.85	0.98	0.98	0.87	0.94	0.88	0.42	0.95	1.00	0.27
	Mistral	0.97	0.69	0.58	0.99	0.81	0.76	0.97	0.74	0.40	0.96	0.82	0.42	0.97	0.91	0.38

**Figure 3: Mean ASR and QSR@3 on C4 and attack ranks across datasets. ASR averages exclude undefined cells; QSR averages include all available cells.**

family. Family conditioning mainly changes the margin, not the top QSR attack on C4.

Finding 2. LLMP is the strongest quality-aware general-scrubbing baseline on C4, with the narrowest margins under UW and KGW.

SynthID: Shared Trend and Model-Level Exceptions. SynthID shares the C4 family-average trend, but its Qwen cell gives SIRA QSR@3 = 0.87 versus 0.85 for LLMP; the Llama values are 0.43 and 0.85. Across these two models, SA removes 186 of 191 recorded pre-positive passages, but only 49 removals pass

Prometheus 3. For LLMP, 178 of 192 recorded pre-positive passages are removed and 170 pass the gate. For SynthID–LLMP, C4 has 192/200 pre-positive records, 178 removals, and 170 gated successes, whereas Dolly has 113/200, 112, and 104. Dolly’s lower joint rate, 0.52 versus 0.85, accompanies near-complete removal of its smaller eligible set.

6.3 RQ3: Utility-Aware Robustness Trade-Offs

RQ3 asks how the recorded joint-success rates change as the automatic acceptance threshold is tightened.

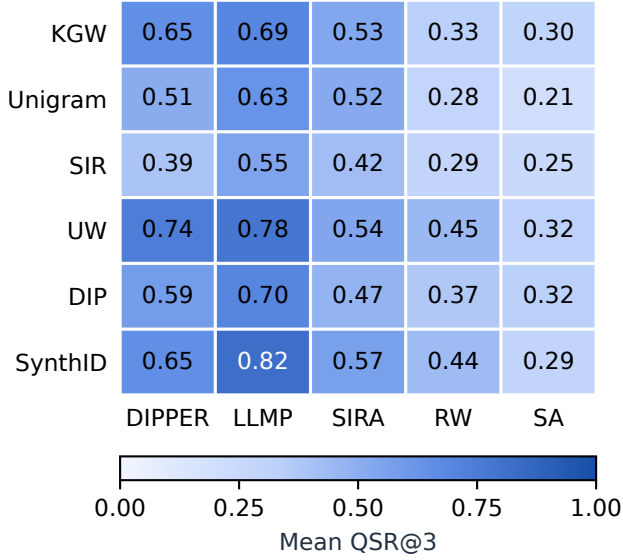


Figure 4: Mean QSR@3 by watermark on C4, averaging available victim-model cells. UW denotes Unbiased.

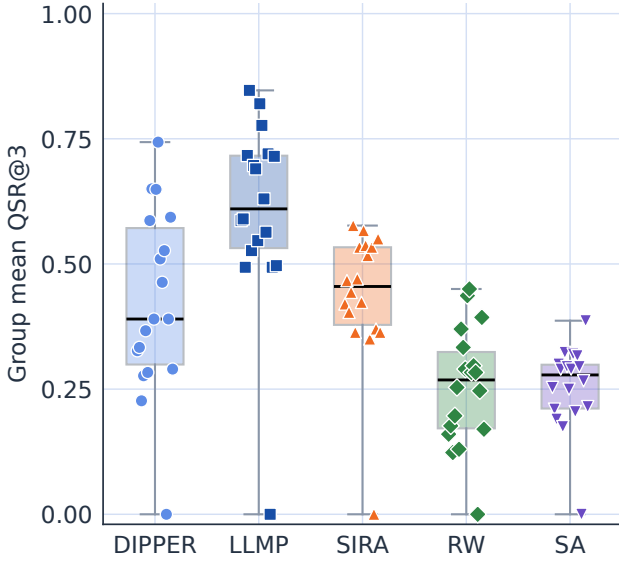


Figure 5: QSR@3 over 18 dataset–watermark groups per attack, each averaged over available models. Points show every group; boxes indicate quartiles and whiskers span the observed range.

For readability, the main text uses two C4 views: threshold sensitivity and attack-conditioned cell variation. Appendix E.2 reports the full per-dataset matrices for detection, QSR, BERTScore, PSP, and Prometheus.

Figure 6 makes the threshold effect explicit. LLMP retains the largest fraction of its QSR under stricter quality thresholds. LLMP

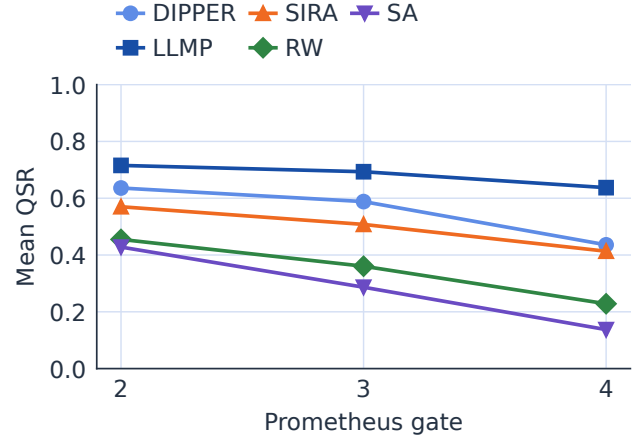


Figure 6: Mean QSR on C4 at Prometheus gates 2, 3, and 4. Each method averages its available victim-model cells.

also remains the strongest attack at each reported threshold. SA degrades much faster. SIRA has the strongest raw ASR, but its quality-aware success drops once the utility gate tightens. DIPPER remains the closest quality-aware alternative to LLMP on C4. The counter-intuitive point is that a stronger detector-side attack can become weaker under a stricter utility gate.

Figure 7 adds a cell-level view. Each point is one watermark–victim-model unit. The plot shows how automatic quality scores and joint success vary across watermark and victim-model conditions. It complements the threshold view by exposing the variation hidden by aggregate attack rankings.

Finding 3. LLMP achieves the highest mean QSR on C4 at all three Prometheus gates.

Cross-Watermark Sensitivity on Common Support. The supplementary analysis fixes Prometheus gates 2, 3, and 4 and uses 180 cells with the same two-model support for all six watermarks and five attacks, keeping the three datasets separate. For SynthID on C4, LLMP’s joint rates are 0.875, 0.850, and 0.770; its margins over the best other attack are 12.11, 16.65, and 24.50 percentage points. LLMP leads each C4 watermark family at all three gates, and each family’s margin is larger at 4 than at 2. SynthID has the largest C4 margin at gate 4. The Qwen exception favors SIRA at gates 2 and 3 but LLMP at gate 4, 0.78 versus 0.76. Table 14 reports the corresponding results for all six watermark families.

6.4 RQ4: Instruction-Tuning Sensitivity

RQ4 asks how the Base–Instruct difference in watermark robustness varies with prompting protocol. We compare Qwen2.5-7B and Qwen2.5-7B-Instruct on C4 at two KGW settings, $h=1, \gamma=0.25$ and $h=1, \gamma=0.5$, using LLMP and SIRA. The *native* condition uses automatic template selection; the *wrapper-disabled* condition omits the chat wrapper. The comparison contains 16 checkpoint–prompting–watermark–attack combinations. Each run uses its

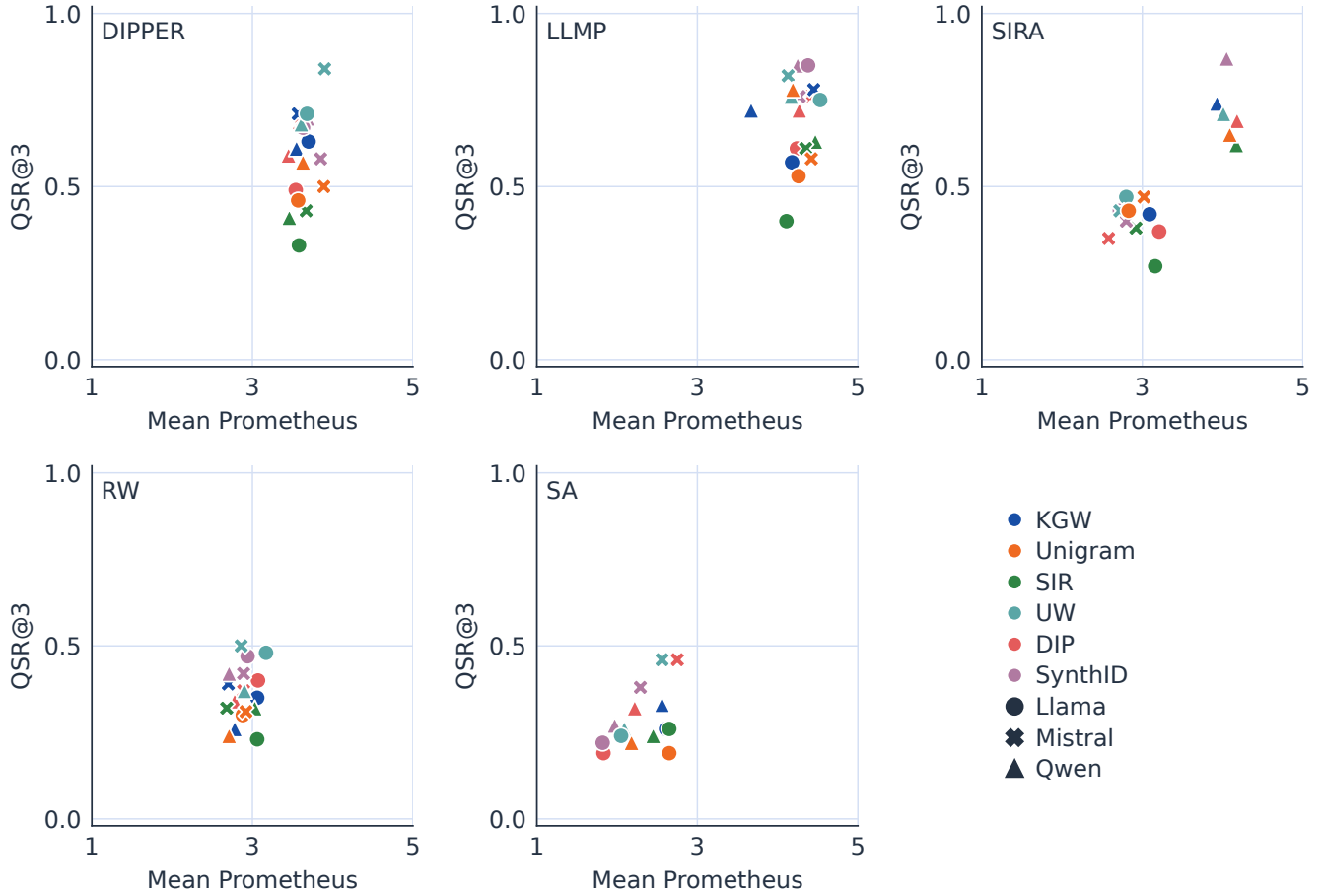


Figure 7: Prometheus scores and QSR@3 on C4. Each point is one victim-model-watermark cell. Color identifies the watermark; shape identifies the model.

generated source texts and recorded detector threshold, so the table reports configuration-level Instruct-minus-Base differences.

Under native prompting, the Instruct-minus-Base pre-detection differences are -0.045 for $\gamma = 0.25$ and -0.015 for $\gamma = 0.5$. For $\gamma = 0.25$, LLMP and SIRA have native QSR differences of $+0.170$ and $+0.160$; disabling the wrapper reduces these descriptive differences to $+0.020$ and $+0.090$. For $\gamma = 0.5$, the native QSR differences are $+0.180$ and $+0.210$, compared with $+0.060$ and $+0.120$ without the wrapper. The corresponding ASR differences change sign in three of four wrapper-disabled entries.

Finding 4. In the evaluated Qwen2.5-7B/KGW runs, the Instruct-Base QSR gap is smaller under wrapper-disabled prompting than under native prompting.

6.5 RQ5: Stealing-Based Attacks

RQ5 asks whether recovered watermark information changes the risk picture beyond the general-scrubbing benchmark. Unlike

RQ1–RQ3, these attacks assume that the attacker first steals or infers reusable watermark information, such as green-list structure, token preferences, or decision statistics. We compare them with strong general-scrubbing baselines, but we do not merge them into the same leaderboard.

The five stealing-based methods share KGW as a target, enabling a comparison across four settings with $h \in \{1, 3\}$, $\gamma \in \{0.25, 0.5\}$, and $\delta = 2$. Mechanistically, h defines how much prior context enters the favored-token rule, while γ defines the green-token base rate. Recovered artifacts guide scrubbing by avoiding or replacing watermark-favored tokens; spoofing-facing use would reverse this direction and favor such tokens during generation. The runs use method-specific helper models, source texts, and recovery budgets. Execution time and recovery resources are reported separately in Table 6.

Table 5 compares representative stealing-based scrubbing methods against the best general-scrubbing baselines on C4. It reports paired-removal ASR, QSR@3, sample counts, Prometheus scores, and differences from the highest general-scrubbing QSR at each setting.

Table 4: Instruct-minus-Base differences on Qwen2.5-7B/KGW under native and wrapper-disabled prompting.

Setting	Protocol	ΔTPR_b	$\Delta\text{ASR}_{\text{LLMP}}$	$\Delta\text{ASR}_{\text{SIRA}}$	$\Delta\text{QSR}_{\text{LLMP}}$	$\Delta\text{QSR}_{\text{SIRA}}$
$h=1, \gamma=0.25$	Native	-0.045	+0.106	+0.046	+0.170	+0.160
$h=1, \gamma=0.25$	Wrapper-disabled	+0.000	-0.070	+0.070	+0.020	+0.090
$h=1, \gamma=0.5$	Native	-0.015	+0.061	+0.059	+0.180	+0.210
$h=1, \gamma=0.5$	Wrapper-disabled	+0.020	-0.028	-0.017	+0.060	+0.120

Table 5: Stealing-based scrubbing on C4/Llama under four KGW settings. General comparators are the highest QSR@3 values among general scrubbers at each setting.

KGW	Method	n	ASR	QSR@3	Best general QSR	ΔQSR	Prom
$h = 1, \gamma = 0.25$	B4	100	0.78	0.72	DIPPER 0.59	+0.13	4.35
	MIP	100	0.17	0.10	DIPPER 0.59	-0.49	3.21
	DE-MARK	100	0.61	0.42	DIPPER 0.59	-0.17	2.91
	SCTS	100	0.22	0.20	DIPPER 0.59	-0.39	4.11
	JSV	100	0.75	0.42	DIPPER 0.59	-0.17	3.01
$h = 1, \gamma = 0.5$	B4	100	0.71	0.60	DIPPER 0.63	-0.03	4.32
	MIP	100	0.33	0.23	DIPPER 0.63	-0.40	3.04
	DE-MARK	100	0.38	0.27	DIPPER 0.63	-0.36	2.97
	SCTS	100	0.26	0.23	DIPPER 0.63	-0.40	4.38
	JSV	99	0.84	0.41	DIPPER 0.63	-0.22	2.89
$h = 3, \gamma = 0.25$	B4	100	0.89	0.78	DIPPER 0.80	-0.02	4.22
	MIP	100	0.72	0.44	DIPPER 0.80	-0.36	3.16
	DE-MARK	100	0.00	0.00	DIPPER 0.80	-0.80	2.79
	SCTS	100	0.49	0.46	DIPPER 0.80	-0.34	4.46
	JSV	100	0.88	0.53	DIPPER 0.80	-0.27	3.05
$h = 3, \gamma = 0.5$	B4	100	0.89	0.83	DIPPER 0.80	+0.03	4.31
	MIP	100	0.78	0.55	DIPPER 0.80	-0.25	3.09
	DE-MARK	100	0.00	0.00	DIPPER 0.80	-0.80	2.64
	SCTS	100	0.39	0.37	DIPPER 0.80	-0.43	4.48
	JSV	99	0.83	0.48	DIPPER 0.80	-0.32	3.13

B4 achieves the highest QSR@3 among the stealing-based scrubbers in all four settings, with rates from 0.60 to 0.83. Its QSR@3 exceeds the best general-scrubbing comparator by 0.13 and 0.03 at two settings and is lower by 0.03 and 0.02 at the other two.

The ASR and QSR comparisons show different outcomes for stealing-based attacks. JSV often approaches the best general-scrubbing baseline in ASR, while its QSR@3 remains lower. SCTS shows the opposite failure mode: it preserves high Prometheus quality, yet its QSR remains below the strongest general-scrubbing baseline because removal is too weak. MIP and DE-MARK are more setting-sensitive and do not provide a consistent gain across the four KGW settings.

Spoofing is the separate case where recovered information is used for false attribution rather than removal. We run a small KGW spoofing feasibility check for JSV-Sp and DE-MARK-Sp, reported in Table 7. QSSR@3 counts forged outputs that trigger the watermark detector and receive Prometheus ≥ 3 against their paired watermarked continuation. The eight spoofing runs each contain 100 outputs; natural-generation detector acceptance is zero in each run. DE-MARK-Sp reaches 98–100% detector acceptance and 47–64% QSSR@3 across the four settings. JSV-Sp reaches 0–1% detector acceptance and zero QSSR@3.

Finding 5. B4 leads stealing-based QSR@3 in all four KGW settings; DE-MARK-Sp attains 98–100% detector acceptance and 47–64% QSSR@3.

6.6 Supplementary Output Screening

We supplement the automatic metrics with output screening against the task and source text. Deterministic rules identify empty and punctuation-only outputs; length reduction and repetition flag texts for Codex-assisted inspection. Across 26,096 outputs, we inspect 251 source–output pairs and identify 254 invalid outputs, including 19 deterministic failures. An invalid output contributes zero to screened QSR while remaining in its original denominator. This excludes 20 QSR@3 successes across 20 cells, and LLMP retains the highest dataset-level mean QSR@3 on C4, Dolly-15K, and BookReport. Appendix E.3 gives the screening criteria, review coverage, and per-method changes.

6.7 Scope And Limitations

The general-scrubbing matrix covers six watermark families, three instruction-tuned model families, and three prompt datasets. The Base–Instruct and recovered-information studies cover KGW under their specified prompting and access conditions. These

configuration-level comparisons use the run-specific sources, detector thresholds, and resource scopes defined in Section 5.2 and Appendix D.

Interpretation and Decision Value. A low joint-success rate can reflect few detectable inputs, low removal among eligible inputs, a strict automatic gate, or missing measurements. These cases lead to different security decisions and should not be collapsed into a watermark ranking. Evaluators should inspect baseline eligibility and paired transitions before choosing attack tests, and verify source identity before pairing methods for uncertainty estimation.

7 Related Work

Recent LLM watermark benchmarks and toolkits have improved the reproducibility of watermark implementation and evaluation. MarkLLM is a representative implementation-oriented toolkit, providing unified interfaces for implementing watermark methods and evaluating generation, detection, quality, and robustness behavior [24]. Complementing this toolkit perspective, WaterBench studies fair comparison of watermark methods under aligned watermarking strength, with particular attention to detection-generation trade-offs and instruction-following quality [32]. From a robustness perspective, WaterPark is the closest prior platform: it integrates a broad suite of removal attacks to evaluate the resilience of LLM watermarks under adversarial perturbations [19]. Other work broadens the evaluation criteria. CEFW proposes a multi-dimensional scoring framework that covers detectability, text quality, usability, robustness, and imperceptibility [39]. MarkMyWords further benchmarks watermark schemes across quality, size, and tamper-resistance under practical attacks [27].

These frameworks are complementary to MarkSec, but they are mostly organized around watermark methods rather than attack methods. In these systems, attacks typically serve as robustness tests, quality-robustness trade-off factors, or components of a composite watermark score. WaterPark is the closest exception because it substantially expands the robustness evaluation. WaterPark also examines semantic preservation and watermark design factors. Its main evaluation target remains the resilience of watermarks under attacks, not a capability-stratified benchmark of attack methods themselves. Prior frameworks also provide limited support for organizing stealing, spoofing, recovered artifacts, and general scrubbing as separate evaluation tracks.

MarkSec addresses this attack-centric gap by asking how attack methods behave under explicit capability assumptions. It treats attack methods as a first-class benchmark dimension. It separates the general-scrubbing comparison from stronger stealing and spoofing analyses. It also reports attack effectiveness together with text quality. This makes MarkSec complementary to watermark implementation and watermark-scoring frameworks: it stress-tests LLM watermarks through modern attack methods and reports the assumptions behind those attacks.

8 Conclusion

We presented MarkSec, a general framework for evaluating adversarial attacks against LLM watermarks under a unified reporting

protocol. The framework is built around an explicit separation between comparable general scrubbing attacks and stronger stealing-based attacks that require additional access, recovered artifacts, or attack-side control. This design allows shared-protocol robustness comparisons while still exposing escalation risks that would be hidden by a text-only evaluation.

Across the evaluated matrix, three findings stand out. First, there is no universal winner across all evaluation axes. Second, family-specific behavior matters: overall averages are useful, but they can hide whether an attack is a broad baseline or a narrow specialist. Third, SIRA leads mean removal on C4, while LLMP leads mean Prometheus-gated joint success across watermark families and retains that lead under stricter gates. Supplementary output screening preserves LLMP’s lead in dataset-level mean QSR@3. The Base-Instruct comparison shows smaller QSR gaps with the prompt wrapper disabled. Under stronger recovered-information access, B4 achieves the highest QSR@3 among the evaluated stealing-based scrubbers in all four KGW settings, and DE-MARK-Sp achieves high detector acceptance with lower quality-gated spoofing success.

More broadly, our results argue for capability-aware watermark evaluation. Stronger attacks should not be ignored, but they should be reported together with the assumptions, query budgets, runtime costs, and artifact provenance that make them possible. We hope MarkSec helps move LLM watermark evaluation toward more reproducible, interpretable, and practically meaningful security assessment.

References

- [1] Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *AISeC@CCS*. 79–90.
- [2] Hongyan Chang, Hamed Hassani, and Reza Shokri. 2025. Watermark Smoothing Attacks against Language Models. In *Findings of EMNLP*. 4915–4941.
- [3] Ruibo Chen, Yihan Wu, Junfeng Guo, and Heng Huang. 2024. De-Mark: Watermark Removal in Large Language Models. *arXiv preprint* (2024).
- [4] Yixin Cheng, Hongcheng Guo, Yangming Li, and Leonid Sigal. 2025. Revealing Weaknesses in Text Watermarking Through Self-Information Rewrite Attacks. In *ICML*. 9982–10009.
- [5] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM. <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>. Introduces the Databricks Dolly 15K dataset and Dolly 2.0.
- [6] Sumanth Dathathri, Abigail See, Shruti Ghaisas, et al. 2024. Scalable Watermarking for Identifying Large Language Model Outputs. *Nature* (2024), 818–823.
- [7] Ashleigh Davis. 2023. ChatGPT Sparks Cheating, Ethical Concerns as Students Try Realistic Essay Writing Technology. <https://www.abc.net.au/news/2023-01-26/chatgpt-sparks-cheating-ethical-concerns-in-schools-universities/101888440>. ABC News, January 25, 2023; accessed 2026-04-26.
- [8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [9] Jocelyn Gecker. 2023. Amid ChatGPT Outcry, Some Teachers Are Inviting AI to Class. <https://apnews.com/article/chatgpt-ai-use-school-essay-7bc171932ff9b994e04f6eae09319f>. AP News, February 14, 2023; accessed 2026-04-26.
- [10] Zhengmian Hu, Lichang Chen, Xidong Wu, Yihan Wu, Hongyang Zhang, and Heng Huang. 2023. Unbiased Watermark for Large Language Models. *arXiv preprint* (2023).
- [11] Baizhou Huang, Xiao Pu, and Xiaojun Wan. 2025. B4: A Black-Box Scrubbing Attack on LLM Watermarks. In *NAACL*. 9113–9126.
- [12] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [13] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (Jan. 2026), 1–72. <https://doi.org/>

- 10.1145/3747588
- [14] Nikola Jovanović, Robin Staab, and Martin Vechev. 2024. Watermark Stealing in Large Language Models. *arXiv preprint* (2024).
 - [15] Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024. Prometheus 2: An Open Source Language Model Specialized in Evaluating Other Language Models. *arXiv preprint arXiv:2405.01535* (2024).
 - [16] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A Watermark for Large Language Models. In *ICML*. 17061–17084.
 - [17] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. 2023. Paraphrasing Evades Detectors of AI-Generated Text, but Retrieval Is an Effective Defense. In *NeurIPS*.
 - [18] Gregory Kang Ruey Lau, Xinyuan Niu, Hieu Dao, Jiangwei Chen, Chuan-Sheng Foo, and Bryan Kian Hsiang Low. 2024. Waterfall: Scalable Framework for Robust Text Watermarking and Provenance for LLMs. In *EMNLP*. 20432–20466.
 - [19] Jiacheng Liang, Zian Wang, Spencer Hong, Shouling Ji, and Ting Wang. 2025. Watermark under Fire: A Robustness Evaluation of LLM Watermarking. In *Findings of EMNLP*. 21050–21074.
 - [20] Weixin Liang, Yaohui Zhang, Mihai Codreanu, Jiayu Wang, Hancheng Cao, and James Zou. 2025. The Widespread Adoption of Large Language Model-Assisted Writing Across Society. arXiv:2502.09747 [cs.CL] <https://arxiv.org/abs/2502.09747>
 - [21] Aiwei Liu, Leyi Pan, Xuming Hu, Shu'ang Li, Lijie Wen, Irwin King, and Philip S Yu. 2023. An Unforgeable Publicly Verifiable Watermark for Large Language Models. *arXiv preprint* (2023).
 - [22] Aiwei Liu, Leyi Pan, Xuming Hu, Shiao Meng, and Lijie Wen. 2023. A Semantic Invariant Robust Watermark for Large Language Models. *arXiv preprint* (2023).
 - [23] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. arXiv:2301.11305 [cs.CL] <https://arxiv.org/abs/2301.11305>
 - [24] Leyi Pan, Aiwei Liu, Zhiwei He, Zitian Gao, Xuandong Zhao, Yijian Lu, Binglin Zhou, Shuliang Liu, Xuming Hu, Lijie Wen, Irwin King, and Philip S. Yu. 2024. MarkLLM: An Open-Source Toolkit for LLM Watermarking. In *EMNLP*. 61–71.
 - [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*. 8024–8035.
 - [26] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590 [cs.SE] <https://arxiv.org/abs/2302.06590>
 - [27] Julien Piet, Chawin Sitawarin, Vivian Fang, Norman Mu, and David Wagner. 2023. Mark My Words: Analyzing and Evaluating Language Model Watermarks. *arXiv preprint* (2023).
 - [28] James Purtill. 2024. Twitter Is Becoming a 'Ghost Town' of Bots as AI-Generated Spam Content Floods the Internet. <https://www.abc.net.au/news/science/2024-02-28/twitter-x-fighting-bot-problem-as-ai-spam-floods-the-internet/103498070>. ABC News, February 27, 2024; accessed 2026-04-26.
 - [29] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
 - [30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* (2020), 1–67.
 - [31] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. 2023. Can AI-Generated Text Be Reliably Detected? *Transactions on Machine Learning Research* (2023).
 - [32] Shangqing Tu, Yuliang Sun, Yushi Bai, Jifan Yu, Lei Hou, and Juanzi Li. 2024. WaterBench: Towards Holistic Evaluation of Watermarks for Large Language Models. In *ACL*. 1517–1542.
 - [33] Shen Wang, Tianlong Xu, Hang Li, Chaoli Zhang, Joleen Liang, Jiliang Tang, Philip S. Yu, and Qingsong Wen. 2024. Large Language Models for Education: A Survey and Outlook. arXiv:2403.18105 [cs.CL] <https://arxiv.org/abs/2403.18105>
 - [34] Tiannan Wang, Jiamin Chen, Qingrui Jia, Shuai Wang, Ruoyu Fang, Huilin Wang, Zhaowei Gao, Chunzhao Xie, Chuou Xu, Jihong Dai, Yibin Liu, Jialong Wu, Shengwei Ding, Long Li, Zhiwei Huang, Xinle Deng, Teng Yu, Gangan Ma, Han Xiao, Zixin Chen, Danjun Xiang, Yunxia Wang, Yuanyuan Zhu, Yi Xiao, Jing Wang, Yiru Wang, Siran Ding, Jiayang Huang, Jiayi Xu, Yilihamu Tayier, Zhenyu Hu, Yuan Gao, Chengfeng Zheng, Yueshu Ye, Yihang Li, Lei Wan, Xinyue Jiang, Yujie Wang, Siyu Cheng, Zhule Song, Xiangru Tang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, Yuchen Eleanor Jiang, and Wangchunshu Zhou. 2024. Weaver: Foundation Models for Creative Writing. arXiv:2401.17268 [cs.CL] <https://arxiv.org/abs/2401.17268>
 - [35] Laura Weidinger, Jonathan Uesato, Maribeth Rauh, Conor Griffin, Po-Sen Huang, John Mellor, Amelia Glaese, Myra Cheng, Borja Balle, Atoosa Kasirzadeh, Courtney Biles, Sasha Brown, Zac Kenton, Will Hawkins, Tom Stepleton, Abeba Birhane, Lisa Anne Hendricks, Laura Rimell, William Isaac, Julia Haas, Sean Legassick, Geoffrey Irving, and Iason Gabriel. 2022. Taxonomy of Risks Posed by Language Models. In *FAccT*. 214–229.
 - [36] Qilong Wu and Varun Chandrasekaran. 2024. Bypassing LLM Watermarks with Color-Aware Substitutions. In *ACL*. 8549–8581.
 - [37] Hanyi Xu, Wensheng Gan, Zhenlian Qi, Jiayang Wu, and Philip S. Yu. 2024. Large Language Models for Education: A Survey. arXiv:2405.13001 [cs.CL] <https://arxiv.org/abs/2405.13001>
 - [38] Hanlin Zhang, Benjamin L. Edelman, Danilo Francati, Daniele Venturi, Giuseppe Ateniese, and Boaz Barak. 2024. Watermarks in the Sand: Impossibility of Strong Watermarking for Language Models. In *ICML*. 58851–58880.
 - [39] Shuhao Zhang, Bo Cheng, Jiale Han, Yuli Chen, Zhixuan Wu, Changbao Li, and Pingli Gu. 2025. CEFW: A Comprehensive Evaluation Framework for Watermark in Large Language Models. In *ICME*. 1–6.
 - [40] Zhaoxi Zhang, Xiaomei Zhang, Yanjun Zhang, Leo Yu Zhang, Chao Chen, Shengshan Hu, Asif Gill, and Shirui Pan. 2024. Stealing Watermarks of Large Language Models via Mixed Integer Programming. In *ACSAC*.

A Open Science

Reproducibility Scope. The reproducibility materials distinguish the manuscript source from the benchmark code, experimental records, and third-party assets.

We view reproducibility as a core requirement for benchmark papers. The main contribution of MarkSec is a reusable evaluation protocol that others can inspect, rerun, and extend. To support that goal, the project is organized around explicit configuration files, versioned benchmark defaults, and shared interfaces for watermark generation, attack execution, detection, quality evaluation, and result persistence.

Artifact Scope. The benchmark repository contains the framework code needed to instantiate the reported evaluation pipeline, including attack adapters, watermark wrappers, detector calibration logic, experiment orchestration, quality-analysis modules, and paper-facing plotting/reporting scripts. The configuration-driven design makes benchmark assumptions auditable: attack settings, watermark parameters, datasets, victim models, and reporting thresholds are defined in named configs rather than buried in ad hoc scripts.

Reproducibility Assets. The evidence manifest identifies TeX and figure dependencies, per-sample records, configurations, and analysis outputs by cryptographic hashes. The experiment records identify the April 2026 runs, September supplementary scoring, and output-screening decisions as separate measurements. The source archive accompanying the PDF is a manuscript archive, not a claim that all third-party models, datasets, or recovered artifacts are redistributable. This is especially important for stealing-based attacks, where recovered artifacts and corpus provenance materially affect interpretation.

Dependencies And Non-Redistributable Components. Some components of the benchmark depend on external model checkpoints, third-party repositories, or datasets that may have their own licenses or access controls. Redistribution of these components is governed by their respective licenses and access conditions; users may need to obtain third-party assets separately under their original terms. The framework is designed to resolve local model paths through repo configuration so that experiments can be rerun on machines with different storage layouts without changing benchmark logic.

Planned Benchmark Release. We intend to release the framework source, configuration files, experiment manifests, and figure-generation scripts needed to reproduce the reported benchmark protocol and extend it with new attacks, watermark families, datasets, and models. This planned release does not include third-party model checkpoints, datasets, or recovered artifacts unless their redistribution terms permit it.

B Ethical Considerations

This paper studies attacks on LLM watermarking systems, which creates an unavoidable dual-use tension. Stronger attack benchmarks are necessary for understanding whether watermarking methods provide meaningful robustness in practice. The same evaluation infrastructure can also lower the barrier to studying or operationalizing watermark removal, stealing, or spoofing strategies. We therefore frame MarkSec as a stress-testing benchmark for defensive evaluation, not as a claim that watermark removal should be optimized without qualification.

Why This Evaluation Is Necessary. Watermarks are often discussed as mechanisms for provenance, platform governance, and misuse mitigation. If their robustness is evaluated only against weak or incomparable baselines, practitioners may overestimate the security they provide. A realistic attack benchmark is therefore important for preventing false confidence, identifying failure modes before deployment, and making defender-side trade-offs visible.

Dual-Use Risk And Mitigation. The highest-risk part of this space is capability-escalated attacks that recover reusable watermark artifacts or support downstream spoofing. For that reason, the paper does not collapse all attacks into a single leaderboard. It reports stronger stealing-based threats separately and emphasizes the assumptions that make them possible, including access patterns, query budgets, runtime costs, and watermark-family alignment. This reporting choice is itself a mitigation: it keeps the analysis focused on security interpretation rather than on decontextualized “best attack” claims.

Release And Use Considerations. Any public release of the benchmark should prioritize reproducibility for defenders and evaluators while avoiding unnecessary concentration of turnkey high-risk assets. In practice, this means documenting assumptions, preserving artifact provenance, and distinguishing benchmark infrastructure from third-party models or recovered attack-side artifacts that may require additional controls or independent reconstruction. Users of the benchmark should interpret the reported attacks as stress tests for robustness claims and should avoid deploying them against systems they do not own or have permission to evaluate.

Broader Impact. Our goal is to improve the reliability of claims about LLM watermark robustness. We believe transparent evaluation with explicit capability assumptions is more responsible than relying on optimistic or incomparable robustness reports, especially for techniques that may inform provenance or policy decisions. At the same time, benchmark results should not be read as evidence that watermarking is either universally sufficient or universally futile. The more responsible conclusion is narrower:

robustness depends on attacker capability, watermark family, deployment format, and the extent to which attacked outputs remain useful after manipulation.

C Generative AI Usage

Generative AI assistance was used for manuscript revision, code inspection and changes, analysis-script development, experiment-artifact analysis, and source–output content inspection.

D Implementation Details

This appendix summarizes the concrete implementation choices behind MarkSec.

D.1 Default Benchmark Configuration

Benchmark Matrix. Unless otherwise noted, the main removal benchmark uses three instruction-tuned victim families: Llama 3.1 8B Instruct, Qwen2.5 7B Instruct, and Mistral 7B Instruct v0.3. We evaluate on C4, Dolly-15K, and MMW BookReport, and randomly sample 100 prompts per dataset. For C4, we prepend an instruction-style continuation prefix so that continuation-style data better matches the chat-oriented victims used in the benchmark.

Watermark Families. The reported benchmark covers six watermark families: KGW, Unigram, SIR, UW, DIP, and SynthID. For KGW stealing-based analysis, we additionally expose an explicit four-setting grid through repo configs with $h \in \{1, 3\}$, $\gamma \in \{0.25, 0.5\}$, and $\delta = 2.0$. Other watermark families are run through the unified framework interface and compared under the same downstream reporting protocol.

Detection And Quality Reporting. Detector calibration targets an FPR of 1%, with decisions taken at the recorded run-specific thresholds. We then report $\text{TPR}_{\text{before}}$, $\text{TPR}_{\text{after}}$, and ASR. For utility-aware reporting, we use Prometheus as the default gating metric and report $\text{QSR}@2$, $\text{QSR}@3$, and $\text{QSR}@4$, with QSR in the main text referring to $\text{QSR}@3$ unless otherwise noted. The paper-facing quality matrices also report BERTScore and PSP as supporting semantic diagnostics.

Sample Universe. Each attack block uses its completed-output population and generated source texts. TPR_b uses available before-detection results; ASR uses before-positive samples with both detector states, and QSR uses all completed outputs (Section 5.2). The result data record the numerator and denominator of each rate and the count of missing predictions.

Attack Defaults. The attack defaults below are repo settings used to instantiate benchmark runs. For the general-scrubbing benchmark, DIPPER uses lexical diversity 60, order diversity 0, sentence interval 1, and a bf16 paraphraser backend; LLMP uses the rewrite prompt template with max new tokens 256, temperature 0.7, and sampling enabled; SIRA uses self-information threshold 30, max new tokens 256, and deterministic decoding, following the official implementation’s percentile-style blanking semantics; RW uses 200 total steps, 40 target valid steps, span length 6, backtrack patience 40, reward-model tie threshold 0.02, and T5 sampling with top- $p = 0.8$, top- $k = 40$, and temperature 1.0; and SA uses $\alpha = 1.0$, top- $k = 10$, 200 confidence-normalization samples, 100 confidence

bins, and max new tokens 200. For protocol-separated stealing-based analysis, MIP uses Gurobi with a 1800s time limit, MIP gap 0.05, AS2 attack mode, 2000 stealing samples, max edit ratio 0.3, and the greedy removal backend by default. SCTS uses context size 1, at most five candidates, query top- $k = 10$, step size 2, max substitution ratio 0.1, and chi-square threshold 0.01. DE-MARK is run against a KGW-style target with context width 1, query batch size 32, sampled token budget 20, predicted/actual $\delta = 2.0$, and query caching enabled. JSV uses a query budget of 30000, batch size 10, previous-context width 3, prebuilt corpus reuse, and downstream scrub/spoof deltas of ± 7.5 . B4 runs in prebuilt-corpus mode with automatic corpus discovery, OPT-IML-1.3B as the default proxy-reference model, contrastive top- $k = 10$, max new tokens 256, and fixed $\lambda = 0.5$.

Capability And Cost Transparency. Table 6 reports attack execution time summed from per-sample durations. Preparation, corpus acquisition, and offline recovery are excluded as specified for each method.

Cost Drivers. MIP’s short online editing time does not include its offline recovery stage. DE-MARK’s spoof rows record about 20,000 token queries, while JSV’s 30,000 figure describes a prebuilt corpus; these quantities are not interchangeable.

D.2 Implementation Notes

Unified Framework Integration. We integrate watermarking methods through the MarkLLM-based interface layer and adapt attack implementations to a shared pipeline for data loading, generation, attack execution, detection, quality analysis, and result persistence.

Direct Removal Versus Stealing-Based Attacks. We intentionally separate general scrubbers from stealing-based attacks in the paper’s reporting. General scrubbers take watermarked text and optional side information as input. Stealing-based attacks first recover a reusable artifact such as a greenlist, token-color map, count table, or proxy distribution. We therefore do not collapse them into a single headline leaderboard.

Local Models And Fixed Checkpoints. All experiments use local checkpoints rather than floating remote model identifiers. This avoids silent version drift during a long-running benchmark campaign and keeps attack-side helper models reproducible under the same local environment.

Prebuilt Stealing Corpora. For stealing-based attacks that depend on large collections of watermarked and natural continuations, the benchmark defaults to canonical prebuilt corpora. These corpora are generated with vLLM V0 rather than V1 because, at the time of writing, the V1 path did not support the custom logits-processor interface required by our watermark injection stack.

Attack Adaptations. SA uses the unified edit pipeline, RW includes prompt-aware cleanup and local quality control, and SIRA’s proxy and rewriting roles share the framework interface. These adaptations connect the attack implementations to the same generation, detection, and reporting components.

E Additional Results

E.1 Family-Level Aggregation for RQ2

RQ2 in the main text summarizes family-conditioned QSR patterns with compact figures. The detailed evidence is the set of appendix tables below. Table 7 reports the limited KGW spoofing feasibility slice used in RQ5. Table 8 and Table 9 report the canonical C4 detection/QSR and attacked-text quality matrices. Table 10 and Table 11 provide the same matrices for Dolly-15K. Table 12 and Table 13 provide the same matrices for MMW BookReport. These tables provide the cell-level values behind the main RQ figures.

E.2 Full General-Scrubbing Matrices

The main text reports compact C4 views for readability and protocol control. This appendix reports the full watermark-victim-model-attack matrices for the canonical C4 slice and two external-validity slices. Detection tables report pre-attack TPR, raw ASR, and QSR@3. The quality tables score attacked text with BERTScore, PSP, and Prometheus. The quality tables report the original run-level automatic measurements. Original per-record BERTScore and PSP measurements are available for 36 cells. For the remaining 225 cells, supplementary scoring in September 2026 evaluates the same 22,497 text pairs; all 450 cell-mean differences from the original scores are below 0.001. The supplementary scores use full-text fields, including a shared prompt when present, and are reported separately in the result data. Appendix E.3 gives the output-screened QSR analysis. Thirteen Dolly-SIR cells have no observable removal opportunities; their ASR is marked “–” and excluded from ASR averages. Unless otherwise noted, QSR uses Prometheus threshold 3. “N/A” denotes combinations excluded by the benchmark protocol. Bold in detection tables marks the row-wise best QSR@3. Bold in quality tables marks the row-wise best Prometheus score.

E.3 Content Validity of Attack Outputs

We screen all 26,096 outputs across the 261 general-scrubbing cells and five attack implementations, with individual source-output inspection of 251 pairs. Empty outputs and outputs containing only punctuation are invalid. Repetition, unrelated responses, and loss of core content require inspection against the source and task; length alone does not establish failure. An invalid output contributes zero to quality-gated success at every gate and remains in the original denominator. The separate screening gate leaves the measured similarity scores and detector states unchanged.

Deterministic checks identify 2 empty and 17 punctuation-only outputs. Source-output inspection assisted by Codex identifies 235 further invalid observations, giving 254 in total. Of the 251 inspected pairs, 16 have no established categorical failure; 25,826 other outputs receive only the mechanical screen. Short verse retaining its core imagery is not rejected for length alone. Some sources are themselves degraded, so output invalidity does not by itself establish attack-caused degradation.

Table 6: Attack execution time across four KGW settings per method. Ranges sum per-sample durations within each run; excluded preparation stages are listed separately.

Method	Objective	Runs	Execution time (min)	Excluded stages
B4	Scrub	4	16.6–18.9	Preparation and corpus acquisition
DE-MARK	Scrub	4	666.0–690.2	Preparation and prior cache construction
JSV	Scrub	4	6.0–13.6	Preparation and corpus acquisition
MIP	Scrub	4	0.09–0.15	Offline recovery and corpus acquisition
SCTS	Scrub	4	2384.7–2731.0	Preparation outside per-sample calls
DE-MARK-Sp	Spoof	4	673.7–701.6	Preparation and prior cache construction
JSV-Sp	Spoof	4	9.3–32.3	Preparation and corpus acquisition

Each run records 100 outputs except two JSV scrub runs with 99. Scoring performed after attack execution is excluded.

Let I_i indicate a confirmed-invalid output, R_i an observed detector removal, and P_i the Prometheus score. The additional descriptive statistic is

$$\text{QSR}_\tau^{\text{screen}} = \frac{1}{|C|} \sum_{i \in C} \mathbf{1}[R_i \wedge P_i \geq \tau \wedge I_i = 0].$$

Here $I_i = 0$ means that invalidity has not been established, rather than a positive content-validity judgment.

This check removes 20 successes across 20 cells at gate 3, 32 at gate 2, and 5 at gate 4. The table summarizes all dataset–attack groups whose gate-3 values change. The denominator and treatment of missing detector states follow Section 5.2.

Dataset	Attack	Removed	QSR@3	Screened
BookReport	DIPPER	3	30.22	30.06
BookReport	RW	2	17.33	17.22
C4	DIPPER	7	58.82	58.43
C4	RW	2	36.06	35.94
Dolly	DIPPER	1	37.72	37.67
Dolly	RW	5	23.17	22.89

Removed counts observations; the last two columns are cell-macro QSR@3 percentages, each over 18 cells. Unlisted groups retain their gate-3 values. LLMP remains the leader of the dataset-level cell macro-averages under this check.

E.4 Spoofing Feasibility

Table 7 reports the limited KGW spoofing feasibility slice used in RQ5. Natural AR is the detector accept rate on ordinary un-watermarked generations. QSSR@3 counts spoofing success only when the forged text is detected as watermarked and reaches Prometheus ≥ 3 against the paired watermarked output. All eight rows have 100 outputs and complete spoof-specific Prometheus scores. The resource columns distinguish token probes for DE-MARK from prebuilt-corpus size for JSV; execution time sums per-sample attack durations.

Table 7: Spoofing feasibility under KGW settings on C4 with Llama-3.1-8B-Instruct.

KGW	Genuine TPR↑	Natural AR↓	Method	SSR↑	QSSR@3↑	Prom↑	Probes / corpus	Exec. time
$h=1, \gamma=0.25$	0.80	0.00	DE-MARK-Sp	1.00	0.47	2.51	20k token	701.6m
$h=1, \gamma=0.25$	0.74	0.00	JSV-Sp	0.00	0.00	2.02	30k corpus	9.3m
$h=1, \gamma=0.5$	0.56	0.00	DE-MARK-Sp	1.00	0.64	2.91	20k token	688.2m
$h=1, \gamma=0.5$	0.64	0.00	JSV-Sp	0.00	0.00	1.95	30k corpus	32.1m
$h=3, \gamma=0.25$	0.71	0.00	DE-MARK-Sp	1.00	0.55	2.80	20k token	674.0m
$h=3, \gamma=0.25$	0.75	0.00	JSV-Sp	0.01	0.00	2.05	30k corpus	32.3m
$h=3, \gamma=0.5$	0.59	0.00	DE-MARK-Sp	0.98	0.56	2.98	20k token	673.7m
$h=3, \gamma=0.5$	0.67	0.00	JSV-Sp	0.01	0.00	1.85	30k corpus	9.4m

Table 8: Full C4 general-scrubbing detection and QSR matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		TPR _b ↑	ASR ↑	QSR@3↑	TPR _b ↑	ASR ↑	QSR@3↑	TPR _b ↑	ASR ↑	QSR@3↑	TPR _b ↑	ASR ↑	QSR@3↑	TPR _b ↑	ASR ↑	QSR@3↑
KGW	Llama	0.93	0.78	0.63	0.92	0.67	0.57	0.93	0.82	0.42	0.96	0.61	0.35	1.00	0.46	0.26
	Qwen	0.96	0.75	0.61	1.00	0.88	0.72	0.96	0.91	0.74	0.99	0.62	0.26	0.95	0.68	0.33
	Mistral	0.94	0.88	0.71	0.97	0.82	0.78	0.94	0.89	0.44	0.95	0.81	0.39	N/A	N/A	N/A
Unigram	Llama	0.72	0.75	0.46	0.77	0.75	0.53	0.72	0.96	0.43	0.73	0.88	0.30	0.73	0.49	0.19
	Qwen	0.81	0.85	0.57	0.86	0.98	0.78	0.81	0.95	0.65	0.80	0.64	0.24	0.75	0.89	0.22
	Mistral	0.90	0.60	0.50	0.81	0.78	0.58	0.90	0.88	0.47	0.91	0.59	0.31	N/A	N/A	N/A
SIR	Llama	0.62	0.62	0.33	0.55	0.78	0.40	0.51	0.84	0.27	0.49	0.71	0.23	0.59	0.76	0.26
	Qwen	0.69	0.79	0.41	0.70	0.96	0.63	0.77	0.95	0.62	0.71	0.76	0.32	0.70	0.91	0.24
	Mistral	0.91	0.55	0.43	0.91	0.69	0.61	0.91	0.73	0.38	0.93	0.80	0.32	N/A	N/A	N/A
UW	Llama	0.90	0.93	0.71	0.85	0.92	0.75	0.90	0.96	0.47	0.87	0.78	0.48	0.90	0.73	0.24
	Qwen	0.84	0.99	0.68	0.83	1.00	0.76	0.84	1.00	0.71	0.75	0.93	0.37	0.82	1.00	0.26
	Mistral	0.95	0.96	0.84	0.94	0.93	0.82	0.95	0.96	0.43	0.98	0.84	0.50	0.95	0.99	0.46
DIP	Llama	0.57	1.00	0.49	0.72	0.90	0.61	0.57	0.93	0.37	0.70	0.89	0.40	0.75	0.97	0.19
	Qwen	0.82	0.94	0.59	0.81	0.96	0.72	0.82	0.96	0.69	0.78	0.88	0.34	0.86	0.99	0.32
	Mistral	0.85	0.94	0.68	0.84	0.95	0.76	0.85	0.92	0.35	0.85	0.85	0.37	0.78	0.99	0.46
SynthID	Llama	0.95	0.80	0.67	0.99	0.89	0.85	0.95	0.87	0.43	0.96	0.86	0.47	0.96	0.95	0.22
	Qwen	0.98	0.85	0.70	0.93	0.97	0.85	0.98	0.98	0.87	0.94	0.88	0.42	0.95	1.00	0.27
	Mistral	0.97	0.69	0.58	0.99	0.81	0.76	0.97	0.74	0.40	0.96	0.82	0.42	0.97	0.91	0.38

Table 9: Full C4 general-scrubbing attacked-text quality matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		BERT↑	PSP↑	Prom↑	BERT↑	PSP↑	Prom↑	BERT↑	PSP↑	Prom↑	BERT↑	PSP↑	Prom↑	BERT↑	PSP↑	Prom↑
KGW	Llama	0.929	0.835	3.70	0.932	0.844	4.18	0.910	0.781	3.09	0.917	0.819	3.06	0.877	0.655	2.61
	Qwen	0.894	0.807	3.55	0.918	0.838	3.67	0.901	0.819	3.93	0.904	0.834	2.78	0.850	0.614	2.56
	Mistral	0.921	0.821	3.57	0.938	0.873	4.45	0.897	0.749	2.79	0.899	0.797	2.70	N/A	N/A	N/A
Unigram	Llama	0.921	0.814	3.57	0.941	0.879	4.26	0.900	0.764	2.83	0.907	0.811	2.88	0.877	0.643	2.65
	Qwen	0.911	0.837	3.63	0.922	0.863	4.19	0.911	0.836	4.09	0.908	0.824	2.71	0.855	0.604	2.18
	Mistral	0.932	0.854	3.89	0.934	0.863	4.42	0.906	0.775	3.02	0.909	0.811	2.92	N/A	N/A	N/A
SIR	Llama	0.929	0.822	3.58	0.936	0.846	4.11	0.910	0.765	3.16	0.915	0.799	3.06	0.880	0.643	2.65
	Qwen	0.900	0.822	3.46	0.918	0.858	4.47	0.905	0.832	4.17	0.908	0.849	3.03	0.851	0.593	2.45
	Mistral	0.924	0.833	3.67	0.935	0.870	4.35	0.897	0.768	2.92	0.902	0.803	2.68	N/A	N/A	N/A
UW	Llama	0.928	0.831	3.68	0.941	0.871	4.53	0.904	0.770	2.80	0.915	0.834	3.17	0.866	0.754	2.05
	Qwen	0.909	0.836	3.61	0.923	0.872	4.17	0.909	0.839	4.01	0.906	0.836	2.90	0.863	0.789	2.09
	Mistral	0.928	0.844	3.90	0.937	0.875	4.13	0.901	0.765	2.72	0.906	0.815	2.86	0.864	0.761	2.56
DIP	Llama	0.928	0.832	3.54	0.940	0.872	4.24	0.906	0.771	3.21	0.919	0.834	3.07	0.864	0.719	1.83
	Qwen	0.905	0.833	3.45	0.920	0.868	4.27	0.910	0.840	4.18	0.909	0.849	2.78	0.865	0.787	2.22
	Mistral	0.925	0.831	3.59	0.940	0.881	4.41	0.898	0.749	2.58	0.908	0.813	2.89	0.862	0.770	2.75
SynthID	Llama	0.929	0.838	3.63	0.940	0.872	4.38	0.904	0.771	2.72	0.914	0.812	2.94	0.867	0.735	1.82
	Qwen	0.903	0.826	3.69	0.917	0.862	4.25	0.905	0.833	4.05	0.906	0.840	2.71	0.862	0.791	1.97
	Mistral	0.929	0.850	3.85	0.939	0.872	4.27	0.896	0.751	2.80	0.908	0.819	2.89	0.862	0.770	2.29

Table 10: Full Dolly-15K general-scrubbing detection and QSR matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑
KGW	Llama	0.77	0.87	0.44	0.81	0.83	0.63	0.77	0.83	0.45	0.82	0.67	0.27	0.77	0.58	0.21
	Qwen	0.85	0.88	0.58	0.90	0.99	0.83	0.85	0.98	0.67	0.85	0.68	0.30	0.87	0.86	0.38
	Mistral	0.85	0.89	0.56	0.87	0.86	0.70	0.85	0.88	0.48	0.80	0.82	0.28	N/A	N/A	N/A
Unigram	Llama	0.39	0.82	0.25	0.42	0.90	0.35	0.39	0.95	0.18	0.42	0.83	0.19	0.36	0.56	0.12
	Qwen	0.92	0.50	0.31	0.93	0.83	0.73	0.92	0.82	0.61	0.91	0.35	0.19	0.93	0.54	0.31
	Mistral	0.56	0.77	0.31	0.48	0.88	0.41	0.56	0.82	0.30	0.51	0.61	0.13	N/A	N/A	N/A
SIR	Llama	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00	0.01	1.00	0.00	0.00	–	0.00
	Qwen	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00
	Mistral	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00	0.00	–	0.00	N/A	N/A	N/A
UW	Llama	0.71	1.00	0.57	0.70	1.00	0.67	0.71	0.97	0.49	0.67	0.97	0.40	0.72	1.00	0.23
	Qwen	0.90	1.00	0.74	0.89	1.00	0.82	0.90	1.00	0.75	0.94	0.84	0.48	0.91	1.00	0.55
	Mistral	0.59	1.00	0.47	0.70	1.00	0.65	0.59	1.00	0.41	0.70	0.91	0.30	0.64	1.00	0.38
DIP	Llama	0.50	0.98	0.36	0.48	1.00	0.47	0.50	0.80	0.21	0.51	0.86	0.29	0.47	0.91	0.10
	Qwen	0.67	0.96	0.44	0.60	1.00	0.58	0.67	0.97	0.47	0.63	0.87	0.26	0.65	1.00	0.37
	Mistral	0.85	0.93	0.59	0.78	0.94	0.64	0.72	0.92	0.37	0.71	0.90	0.34	0.78	0.96	0.48
SynthID	Llama	0.49	0.96	0.39	0.46	0.98	0.44	0.49	0.94	0.28	0.51	0.86	0.22	0.50	1.00	0.17
	Qwen	0.61	0.93	0.40	0.67	1.00	0.60	0.61	1.00	0.48	0.63	0.87	0.37	0.58	1.00	0.28
	Mistral	0.51	0.98	0.38	0.47	0.98	0.44	0.51	0.98	0.35	0.42	0.88	0.15	0.53	1.00	0.35

Table 11: Full Dolly-15K general-scrubbing attacked-text quality matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑
KGW	Llama	0.910	0.818	3.10	0.939	0.882	4.47	0.897	0.758	3.49	0.915	0.850	3.16	0.872	0.677	2.68
	Qwen	0.905	0.801	3.18	0.928	0.856	4.32	0.894	0.778	3.80	0.910	0.854	2.99	0.865	0.651	2.54
	Mistral	0.914	0.826	3.19	0.932	0.861	4.43	0.895	0.755	3.32	0.894	0.772	2.74	N/A	N/A	N/A
Unigram	Llama	0.910	0.810	3.24	0.938	0.869	4.41	0.893	0.756	3.15	0.921	0.868	3.30	0.871	0.683	2.99
	Qwen	0.905	0.810	3.25	0.926	0.850	4.40	0.899	0.786	3.71	0.911	0.844	3.09	0.863	0.652	2.70
	Mistral	0.919	0.840	3.39	0.928	0.856	4.27	0.897	0.755	3.12	0.901	0.792	2.87	N/A	N/A	N/A
SIR	Llama	0.910	0.813	3.10	0.938	0.875	4.48	0.898	0.766	3.34	0.912	0.856	2.82	0.870	0.666	2.99
	Qwen	0.898	0.783	3.15	0.924	0.853	4.44	0.897	0.782	3.76	0.919	0.870	3.37	0.862	0.641	2.83
	Mistral	0.915	0.832	3.21	0.929	0.857	4.41	0.890	0.734	3.36	0.902	0.802	2.81	N/A	N/A	N/A
UW	Llama	0.916	0.836	3.22	0.941	0.890	4.49	0.904	0.790	3.23	0.920	0.868	2.96	0.857	0.756	1.86
	Qwen	0.909	0.817	3.35	0.925	0.851	4.42	0.895	0.771	3.73	0.916	0.867	3.07	0.889	0.845	2.86
	Mistral	0.919	0.844	3.39	0.931	0.860	4.33	0.894	0.751	3.39	0.906	0.820	2.68	0.884	0.833	2.99
DIP	Llama	0.914	0.833	3.19	0.936	0.877	4.44	0.903	0.787	3.29	0.924	0.875	3.07	0.858	0.759	1.80
	Qwen	0.903	0.784	3.19	0.928	0.856	4.38	0.897	0.772	3.80	0.906	0.831	2.98	0.889	0.848	2.63
	Mistral	0.915	0.825	3.13	0.931	0.862	4.12	0.890	0.737	3.03	0.905	0.811	2.91	0.881	0.828	2.99
SynthID	Llama	0.911	0.808	3.41	0.938	0.878	4.36	0.901	0.781	3.33	0.917	0.859	3.00	0.858	0.765	2.03
	Qwen	0.902	0.779	3.11	0.927	0.848	4.44	0.898	0.781	3.79	0.913	0.840	3.16	0.884	0.837	2.63
	Mistral	0.920	0.837	3.33	0.931	0.859	4.34	0.896	0.753	3.25	0.905	0.808	2.95	0.882	0.827	2.90

Table 12: Full MMW BookReport general-scrubbing detection and QSR matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑	TPR _b ↑	ASR ↑	QSR@3 ↑
KGW	Llama	0.90	0.97	0.40	0.93	0.69	0.55	0.90	0.90	0.38	0.93	0.76	0.30	0.94	0.59	0.24
	Qwen	0.99	0.89	0.24	0.98	0.97	0.56	0.99	0.94	0.68	0.97	0.74	0.13	1.00	0.60	0.18
	Mistral	0.90	0.99	0.36	0.89	0.83	0.66	0.90	0.98	0.34	0.95	0.89	0.10	N/A	N/A	N/A
Unigram	Llama	0.74	0.69	0.27	0.71	0.65	0.44	0.74	0.96	0.33	0.67	0.85	0.19	0.76	0.37	0.13
	Qwen	0.98	0.69	0.28	0.98	0.81	0.69	0.98	0.72	0.53	0.93	0.59	0.16	0.96	0.48	0.22
	Mistral	0.76	0.88	0.30	0.57	0.89	0.45	0.76	0.92	0.23	0.71	0.55	0.04	N/A	N/A	N/A
SIR	Llama	0.68	0.71	0.25	0.65	0.77	0.45	0.68	0.87	0.38	0.54	0.74	0.12	0.59	0.80	0.16
	Qwen	0.90	0.72	0.26	0.88	0.98	0.52	0.90	0.87	0.53	0.67	0.76	0.17	0.59	0.90	0.22
	Mistral	0.77	0.66	0.17	0.82	0.68	0.51	0.77	0.94	0.30	0.84	0.70	0.08	N/A	N/A	N/A
UW	Llama	0.99	0.98	0.40	1.00	0.91	0.83	0.99	0.96	0.58	0.99	0.72	0.28	0.98	0.32	0.07
	Qwen	1.00	1.00	0.34	1.00	1.00	0.86	1.00	1.00	0.81	1.00	0.83	0.39	1.00	0.90	0.35
	Mistral	1.00	0.95	0.36	1.00	0.97	0.85	1.00	0.94	0.34	1.00	0.88	0.09	1.00	0.80	0.45
DIP	Llama	0.50	0.98	0.27	0.44	1.00	0.39	0.50	0.96	0.27	0.34	1.00	0.08	0.37	1.00	0.03
	Qwen	0.82	0.99	0.45	0.89	0.93	0.66	0.82	0.99	0.75	0.87	0.87	0.32	0.89	0.99	0.46
	Mistral	0.68	1.00	0.26	0.83	0.95	0.71	0.69	1.00	0.24	0.70	0.96	0.08	0.80	1.00	0.27
SynthID	Llama	0.54	0.93	0.20	0.82	0.94	0.71	0.54	0.93	0.26	0.75	0.93	0.25	0.77	0.97	0.15
	Qwen	0.98	0.83	0.31	0.99	0.94	0.74	0.98	0.92	0.71	0.98	0.78	0.26	1.00	0.87	0.40
	Mistral	0.89	0.84	0.32	0.85	0.92	0.70	0.89	0.91	0.36	0.87	0.90	0.08	0.91	0.90	0.35

Table 13: Full MMW BookReport general-scrubbing attacked-text quality matrix.

Watermark	Model	DIPPER			LLMP			SIRA			RW			SA		
		BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑	BERT ↑	PSP ↑	Prom ↑
KGW	Llama	0.900	0.814	2.26	0.945	0.913	4.05	0.874	0.681	2.56	0.899	0.796	2.26	0.881	0.741	2.43
	Qwen	0.888	0.763	1.79	0.928	0.878	3.19	0.885	0.754	3.33	0.893	0.784	1.71	0.872	0.724	1.94
	Mistral	0.898	0.786	2.27	0.932	0.885	4.15	0.866	0.650	2.28	0.856	0.636	1.35	N/A	N/A	N/A
Unigram	Llama	0.892	0.787	2.39	0.944	0.903	4.22	0.869	0.660	2.27	0.894	0.777	2.14	0.880	0.753	2.37
	Qwen	0.886	0.750	2.16	0.929	0.876	3.94	0.882	0.743	3.51	0.903	0.822	2.10	0.871	0.723	2.26
	Mistral	0.898	0.795	2.21	0.936	0.892	4.17	0.862	0.633	2.07	0.873	0.702	1.59	N/A	N/A	N/A
SIR	Llama	0.900	0.814	2.40	0.947	0.909	4.12	0.878	0.694	2.96	0.900	0.818	2.04	0.881	0.743	2.47
	Qwen	0.892	0.792	2.09	0.924	0.863	3.10	0.884	0.751	3.20	0.895	0.789	2.01	0.875	0.735	2.15
	Mistral	0.897	0.785	2.20	0.932	0.891	4.15	0.872	0.676	2.18	0.866	0.673	1.53	N/A	N/A	N/A
UW	Llama	0.902	0.830	2.28	0.947	0.910	4.10	0.876	0.702	2.87	0.899	0.797	2.26	0.859	0.802	1.56
	Qwen	0.888	0.767	2.00	0.926	0.862	3.89	0.881	0.746	3.73	0.905	0.829	2.43	0.900	0.882	2.18
	Mistral	0.900	0.791	2.16	0.935	0.889	4.11	0.867	0.648	2.11	0.861	0.662	1.36	0.890	0.864	2.62
DIP	Llama	0.900	0.812	2.44	0.944	0.905	4.12	0.874	0.688	2.75	0.898	0.799	2.10	0.860	0.782	1.43
	Qwen	0.893	0.783	2.43	0.928	0.868	3.78	0.885	0.754	4.00	0.901	0.821	2.33	0.897	0.877	2.53
	Mistral	0.900	0.789	2.19	0.934	0.895	4.01	0.869	0.663	2.08	0.864	0.656	1.46	0.882	0.856	2.18
SynthID	Llama	0.899	0.819	2.26	0.942	0.903	4.30	0.873	0.677	2.60	0.901	0.818	2.29	0.860	0.793	1.58
	Qwen	0.887	0.771	2.09	0.925	0.868	3.75	0.882	0.748	3.53	0.898	0.814	2.23	0.899	0.881	2.54
	Mistral	0.903	0.805	2.32	0.939	0.898	3.99	0.877	0.686	2.51	0.865	0.686	1.51	0.891	0.870	2.45

F Cross-Watermark Quality-Gate Sensitivity

SynthID configuration. The evaluated MarkLLM SynthID-Text implementation uses iterative probability reweighting with 30 keyed binary g -value channels over five-token n -grams, including four preceding context tokens. Its configuration uses a 65,536-entry sampling table, table seed 0, and a 1,024-context repetition history. The mean detector uses the victim tokenizer without added special tokens and excludes repeated contexts and positions from EOS onward. The stored z_score field contains the mean g -value.

All 45 SynthID runs use dynamic detection with increasing-score decisions and target calibration FPR 0.01. The run-specific threshold is selected from natural and watermarked score populations and applied as $score \geq \tau$; thresholds range from 0.5099 to 0.5228. Configuration and implementation identifiers accompany the result data.

Analysis population. The analysis uses 180 cells: three datasets, six watermarks, Llama and Qwen, and five attacks. These two victim models provide all five attacks for every watermark; Mistral has structural attack exclusions and remains in the full benchmark tables. Each entry averages the Llama and Qwen cell rates with equal weight at Prometheus gates 2, 3, and 4, using the QSR denominator in Section 5.2. The 17,996 outputs include 33 with at least one missing detector prediction; these outputs remain in the QSR denominator and receive no success credit.

Table 14: Quality-gate sensitivity on common Llama/Qwen support.

Dataset	Watermark	LLMP QSR@ u (%)			LLMP minus best other (pp)		
		$u = 2$	$u = 3$	$u = 4$	$u = 2$	$u = 3$	$u = 4$
C4	KGW	68.00	64.50	56.00	+2.00 (S)	+2.50 (D)	+6.00 (S)
	Unigram	68.00	65.50	59.50	+8.00 (S)	+11.50 (S)	+12.50 (S)
	SIR	53.00	51.50	48.50	+6.50 (S)	+7.00 (S)	+12.00 (S)
	UW	77.00	75.50	70.00	+0.50 (D)	+6.00 (D)	+16.50 (D)
	DIP	70.00	66.50	60.50	+9.00 (D)	+12.50 (D)	+15.00 (S)
	SynthID	87.50	85.00	77.00	+12.11 (D)	+16.65 (D)	+24.50 (S)
Dolly-15K	KGW	75.50	73.00	69.50	+14.50 (S)	+17.00 (S)	+21.00 (S)
	Unigram	55.50	54.00	49.50	+12.00 (S)	+14.50 (S)	+16.00 (S)
	SIR	0.00	0.00	0.00	+0.00 (T)	+0.00 (T)	+0.00 (T)
	UW	75.77	74.74	71.66	+2.77 (D)	+9.24 (D)	+23.16 (S)
	DIP	53.00	52.50	48.00	+7.00 (D)	+12.50 (D)	+18.50 (S)
	SynthID	52.50	52.00	50.00	+9.50 (D)	+12.50 (D)	+18.50 (S)
BookReport	KGW	62.50	55.50	49.00	+5.00 (S)	+2.50 (S)	+9.50 (S)
	Unigram	57.00	56.50	50.00	+10.00 (S)	+13.50 (S)	+18.00 (S)
	SIR	52.50	48.50	42.50	+3.50 (S)	+3.00 (S)	+7.50 (S)
	UW	87.50	84.50	74.50	+11.00 (S)	+15.00 (S)	+22.00 (S)
	DIP	56.50	52.50	44.50	+3.50 (S)	+1.50 (S)	+3.00 (S)
	SynthID	76.50	72.50	62.50	+24.50 (S)	+24.00 (S)	+27.00 (S)

Reading the table. Rates are percentages; differences are percentage points relative to the highest-rate non-LLMP attack at that gate. D denotes DIPPER and S denotes SIRA; T denotes a tie among all four non-LLMP attacks. The comparator is selected separately at each gate. The result data provide per-cell denominators, model-specific rates, and all five attack rankings.